

CTF 网络安全入门

Reverse 专攻版

把“程序说不”改写成等式 • 从黑盒观察到白盒验证

本册特色

- 零基础起点：环境安装、命令行、字节与十六进制，每个术语首现都有白话解释；
- 三道带做题（R1 异或校验器、R2 乘加校验器、玄机 589 闸机票根），每题配真实终端截图；
- 每张截图都来自真实执行：命令输出先被录制为实录文本，再渲染成终端窗口图，可按命令逐条复现；
- 工具箱逐个演示：file、strings、xxd、objdump、readelf、python3 字节运算；
- 附录含命令速查表、术语表与玄机命令速查表与术语表。

适用对象：学过 Python、C 和命令行的 CTF 初学者

版本：2026 年 9 月（零基础重构版）

使用说明

这本讲义教什么

Reverse（逆向工程）方向的题目，通常给你一个可执行程序（或一段编译后的代码），程序只回答“对”或“不对”。你的任务是弄清它内部的判定逻辑，反推出它接受的输入，或直接解出它藏起来的 flag。本册按“先铺垫、再工具、再题目、再带做”的顺序展开，学完你应该能独立完成同类型的入门逆向题。

学习路线图

1. **第 1 章零基础预备**：装好 WSL 与工具，认识命令行的管道与重定向；
2. **第 2 章基础知识铺垫**：程序怎么从源码变成机器码，x86-64 汇编与寄存器、栈帧与调用约定，二进制里能看到什么，逆向的标准路线，字节运算与 RC4 概念；
3. **第 3 章工具箱**：六个常用工具，每个都有真实输出截图；
4. **第 4 章题目：题面卡与逐题图解带做**：三道题每题一张题面卡，卡后面直接就是跟着截图一步步做的解题过程；
5. **第 6 章复盘**：知识地图、易错清单、下次怎么办；
6. **第 7 章自测与参考答案**：先做题再看推演；
7. **附录**：命令速查表、术语表。

怎么用这本讲义

建议先只读第 4 章的题面卡自行尝试，再对照同一题后面的解题步骤、配套源码和脚本复现。讲义中每条命令都注明执行环境（Windows PowerShell 还是 WSL）。**截图说明**：所有终端截图均由工具真实执行命令后录制渲染，未做任何内容修改；**原理示意图**（流程图、对应关系

图) 为 matplotlib 绘制的教学示意图, 图注中会注明。玄机平台 (xj.edisec.net) 的页面无法截图, 凡涉及平台处均以文字表格示意, 并注明“示意, 非平台截图”。

安全声明

所有实验只针对本资料附带的程序和明确允许练习的平台题目。不要把命令用于未经授权的系统; 来源不明的二进制先做静态分析, 不要随手运行; 不要把账号密码写进讲义、脚本或共享压缩包。

术语约定

每个术语第一次出现时都会用白话解释, 并在附录 C 术语表中给出一句话定义。**字节** (byte) 是计算机存储的最小可寻址单位, 一个字节有 8 个**位** (bit), 能表示 0–255 共 256 个数值; 本册几乎所有的“魔法数” (如 0x23、171) 都是字节层面的数值。**十六进制** (hex) 是人类看字节的常用记法, 两位数字表示一个字节, 如 0x45 表示数值 69。

已验证答案一览 (原讲义事实原样保留)

本地题 R1 的接受输入为 `flag{reverse_xor}`, R2 为 `flag{trace_the_loop}`; 玄机 589 闸机票根的票根与 flag 由本册解题脚本真实运行得到 (见第 4 章图 4.13); 原讲义已验证的玄机 557 qgd 答案为 `flag{933de8e7e0/3b162f64bcfec94803d}`, 相关推演原样保留在第 7 章与附录 A。除上述内容外, 本册不引入任何新的 flag。

目录

使用说明	i
第一章 零基础预备：实验环境、命令行与工具安装	1
1.1 逆向是什么：用白话说清楚	1
1.2 实验环境：Windows + WSL	2
1.3 工具安装与验证	2
1.4 命令行的三个小概念：管道、重定向、退出码	4
1.5 好习惯：先观察，后动手	5
第二章 Reverse 基础知识铺垫	6
2.1 程序从源码到可执行文件	6
2.2 源码、汇编、机器码：一件事的三种写法	7
2.2.1 读汇编前的第一课：寄存器是 CPU 手边的小格子	8
2.2.2 十几条够用的最小指令集	9
2.2.3 AT&T 与 Intel：同一条指令的两种写法	9
2.2.4 函数怎么被调用：栈、栈帧与参数传递	11
2.3 二进制里能看到什么：字符串、常量、代码逻辑	11
2.4 逆向的标准路线：黑盒观察 → 白盒验证	12
2.5 字节、ASCII 与十六进制	13
2.6 异或 (XOR)：自己的逆运算	14
2.7 Python 字节运算反推：从 TARGET 逆出 $(b*3+7)\&255$	15
2.8 RC4 的两个阶段：KSA 与 PRGA	16
2.8.1 为什么 589 要“先解票根再解 flag”	18
2.9 本章小结：把术语串成一条线	18

第三章 Reverse 工具箱	19
3.1 file: 先问“这是什么”	19
3.2 strings: 翻一翻可读文字	20
3.3 xxd: 用十六进制看字节	21
3.4 objdump -d: 反汇编看逻辑	22
3.5 readelf -h: 看 ELF 身份证	23
3.6 python3: 字节运算的计算器	24
3.7 本章工具一览	25
第四章 题目: 题面卡与逐题图解带做	26
4.1 题 R1: 程序说 nope, 正确输入长什么样?	27
4.2 题 R2: 乘 3 加 7 的循环, 怎么倒着走?	33
4.3 题 R2+: 玄机 589 闸机票根——票根是什么, flag 又藏在哪儿?	37
4.4 三道题的共性复盘	41
第五章 复盘: 知识地图与易错清单	42
5.1 知识地图	42
5.2 易错清单	42
5.3 下次遇到这类题怎么办	43
第六章 自测与参考答案	45
6.1 自测题	45
6.2 参考要点	45
6.3 完整推演一: R1 从目标数组还原输入	46
6.4 完整推演二: R2 从目标列表还原输入	46
6.5 完整推演三: 589 从.rodata 到两行输出	47
6.6 保留记录: 玄机 557 qgd 的完整推演(原讲义结论原样保留)	47
6.7 自测评分参考	48
附录 A 命令速查表	49
A.1 观察与识别	49
A.2 反汇编与数据段	49
A.3 运行与破解	50

第一章 零基础预备：实验环境、命令行与工具安装

本章目标：在自己的电脑上搭好“能跑 Linux 程序、能看二进制”的实验环境，并养成先观察、后动手的习惯。

1.1 逆向是什么：用白话说清楚

先说人话：朋友给你一个小程序，说“输入正确的口令才放行”。你运行它，随便输什么都显示 `nope`。逆向工程（reverse engineering，常简称 reverse 或 RE）就是：在看不到原始源码的情况下，弄明白这个程序到底怎么判断“口令正确”，从而算出正确口令。

CTF 里的 Reverse 题有三个常见形态：

- 给你一个可执行文件（Linux 的 ELF 或 Windows 的 EXE），要你找出被接受的输入，输入本身就是 flag；
- 给你一段编译后的代码或字节，要你还原它做的事，算出 flag；
- 给你一个加了“壳”的程序（比如 PyInstaller 打包的 Python 程序），要你拆开包装，找到里面的加密逻辑。

逆向题的“对手”不是别人，是编译器。编译器把你看得懂的源码翻译成了机器码；我们要把这趟翻译反着走回去。**编译器**（compiler）是把 C 等高级语言翻译成 CPU 能执行的机器码的程序；**机器码**（machine code）就是一堆 0/1 字节，CPU 逐条执行。

一句话记忆

逆向 = 在没有源码的前提下，从可执行文件还原“程序的判定逻辑”，并算出它接受的输入。

1.2 实验环境：Windows + WSL

本册的练习程序是 Linux ELF 文件，需要在 Linux 环境运行。Windows 用户最省事的办法是安装 **WSL**（Windows Subsystem for Linux，Windows 的 Linux 子系统）：在 PowerShell（管理员）里执行 `wsl --install -d Ubuntu-24.04`，装完后从开始菜单打开 Ubuntu 即可。详细的安装步骤和常见问题请看 README.md；Linux 虚拟机同样可用，命令一致。

Windows 与 WSL 的路径写法不同：Windows 的 C:\ 盘在 WSL 中通常位于 `/mnt/c/`。本资料放在

```
C:\Users\你的用户名\Desktop\CTF零基础自学资料包
```

对应 WSL 路径

```
/mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包
```

终端的当前目录以资料根目录为准，动手前先用 `pwd` 和 `ls` 确认位置，避免“找不到文件”式的空转。

常见误区

Windows 的记事本、PowerShell 和 WSL 终端是三个不同的世界。截图里形如 `mzj@Ubuntu-24.04:~$` 的提示符（`mzj` 是作者电脑上的用户名，你机器上会显示你自己的用户名）表示“这条命令在 WSL 里跑”；带 `PS` 开头提示符的才是 PowerShell 命令。

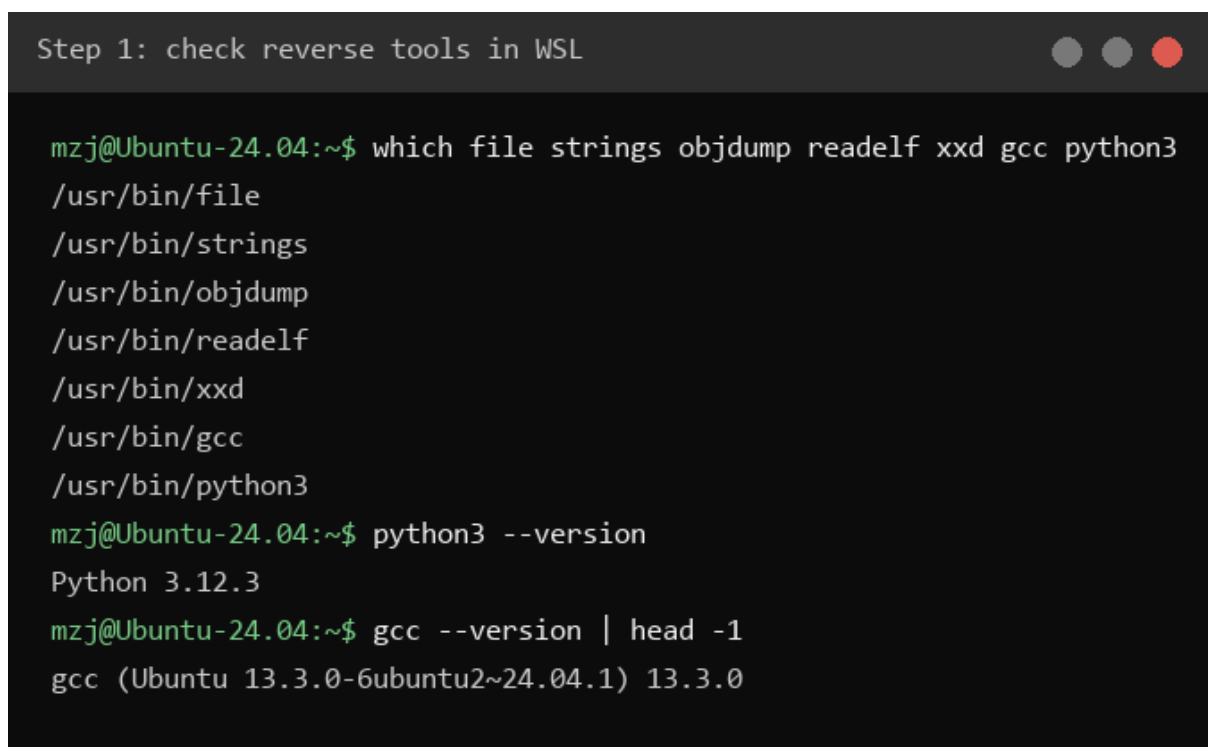
1.3 工具安装与验证

逆向入门只需要六个小工具，WSL/Ubuntu 下用一条命令就能装齐：

```
sudo apt update && sudo apt install -y binutils file bsdextrautils gcc python3
```

其中 `binutils` 提供 `objdump` 与 `readelf`，`file` 识别文件类型，`bsdextrautils` 提供 `xxd`，`gcc` 用来编译练习程序，`python3` 用来写字节运算脚本。装完用 `which` 逐个确认存在：`which` 的作用是打印某个命令的安装路径，什么都没打印就说明没装上。

图 1.1 是本册实际执行的验证结果（真实终端截图）：七个工具全部就位，`python3` 为 3.12.3，`gcc` 为 13.3.0。你的版本号略有差别没关系，只要有输出即可。



```
Step 1: check reverse tools in WSL

mzj@Ubuntu-24.04:~$ which file strings objdump readelf xxd gcc python3
/usr/bin/file
/usr/bin/strings
/usr/bin/objdump
/usr/bin/readelf
/usr/bin/xxd
/usr/bin/gcc
/usr/bin/python3
mzj@Ubuntu-24.04:~$ python3 --version
Python 3.12.3
mzj@Ubuntu-24.04:~$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
```

图 1.1: 真实终端截图: 用 `which` 确认 `file` / `strings` / `objdump` / `readelf` / `xxd` / `gcc` / `python3` 全部安装成功, 并查看 `python3` 与 `gcc` 版本

先把练习程序编译出来 后面第 3 章和第 4 章会反复用到两个练习文件: `check1` (一个 C 程序, 需要编译) 和 `verify2.py` (Python 脚本, 直接运行)。`check1.c` 随包提供, 但二进制 `check1` 要你自己编——现在编好, 后面就不用回头补。在 WSL 里进入资料根目录后执行:

```
cd /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包
gcc -O0 -o labs/reverse/check1 labs/reverse/check1.c
file labs/reverse/check1
```

图 1.2 是真实执行结果: `file` 认出它是 ELF 64-bit LSB pie executable, x86-64 ... not stripped, 看到这行就说明编译成功。not stripped 意思是符号表还在, 所以第 3 章的 `strings`、`objdump` 能直接读出函数名。

```
Step 1: compile check1.c, then confirm with file

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF && gcc -O0 -o labs/reverse/check1
labs/reverse/check1.c && file labs/reverse/check1
labs/reverse/check1: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=81dbd74bf545fdb811d9406955e4424bcfc1e4f,
for GNU/Linux 3.2.0, not stripped
```

图 1.2: 真实终端截图: 在 WSL 里用 `gcc -O0` 编译 `check1.c`, 再用 `file` 确认产物是 ELF 64-bit、not stripped (终端实录截图)

常见误区

①报 No such file or directory: 当前目录不对。先 `pwd` 看看自己在哪, 资料根目录里应该能看到 `labs/`、`figures/`、`README.md`。②报 No rule to make target 或找不到 `check1.c`: 路径拼错了, 确认 `ls labs/reverse/` 能列出 `check1.c`。③后面忘了自己编过、结果 `xxd` 提示文件不存在: 回到本页重新执行一次这三条命令。

常见误区

为什么固定用 `-O0`?

`-O0` 表示“不做优化”, 编译器把你写的代码按原样顺序翻译成机器码。加了优化后, 编译器可能把校验循环合并、改写甚至整段删掉, 反汇编出来的样子就和讲义里展示的不一样了。练习题一律用 `-O0`, 保证“看到的汇编 = 写的代码”。

`python3` 有两种用法, 第 2、3 章的示例都会出现, 先认识一下: 把代码写进文件后 `python3` 脚本 `.py` 一次跑完; 或者直接敲 `python3` 回车, 进入交互式解释器 (REPL, read-eval-print loop, 读一行、算一行、打印一行)。进入后提示符变成 `>>>`, 你输入一个表达式它立刻给出结果——第 2.5 节和 3.6 节那些以 `>>> 0x45` 开头的代码块, 就是在画这个界面。想退出时按 `Ctrl+D` (或输入 `exit()`)。本书里凡出现 `>>>` 的行, 都在提示你“这一句可以直接敲进 REPL 验证”。

1.4 命令行的三个小概念: 管道、重定向、退出码

带做步骤会大量使用下面三个写法, 先在这里认识:

- **管道 (pipe)**: 竖线 `|`, 把左边命令的输出当作右边命令的输入。`echo hello | ./check1` 的意思是“把字符串 `hello` 喂给程序 `check1`”, 相当于你在键盘上敲了 `hello` 再回车。`echo` 命令负责打印一行文字, `./check1` 表示运行当前目录下名为 `check1` 的文件。
- **标准输入/输出 (stdin/stdout)**: 程序默认从键盘读入 (stdin)、向屏幕打印 (stdout)。用管道可以把“键盘”换成“另一个命令的输出”。

- **退出码** (exit code): 命令结束后留下的一个数字, 0 通常表示成功。用 `echo $?` 可以看到上一条命令的退出码。

另一个常见写法是 `>` 重定向, 如 `ls > list.txt` 把输出写进文件而不是屏幕。明白这三点, 后面截图里的每条命令你都能看懂“输入从哪来、输出到哪去”。

一句话记忆

命令之间用 `|` 接流水线: `echo 输入 | 程序` 就是“喂输入给程序”的标准姿势。

1.5 好习惯：先观察，后动手

拿到陌生材料, 第一个动作永远是观察, 而不是运行:

1. 先回答两个问题: “这题要我交付什么?” “程序正常 (未破解时) 的输出是什么?” ——后者叫**基线** (baseline), 是判断“有没有变化”的参照;
2. 用 `file` 看它是什么类型的文件;
3. 用 `strings` 翻一翻有没有可读的提示;
4. 再决定是静态分析 (读文件) 还是运行观察 (喂输入)。

来源不明的 Windows EXE 更要谨慎: 先静态分析, 必要时在受控的虚拟机里运行。考试平台给出的练习附件可以放心在练习环境里分析, 但“先静态后动态”的顺序不变。

小技巧

每一步的结论都写成一句可复现的话: “我执行了命令 X, 看到输出 Y, 因此认为 Z”。这条证据链就是逆向题的解题报告, 也是队友复核的依据。

第二章 Reverse 基础知识铺垫

本章目标：理解程序从源码到可执行文件的全过程，知道字符串、常量和代码逻辑分别藏在二进制的哪里，并掌握字节运算反推与 RC4 的基本概念。

2.1 程序从源码到可执行文件

写完 `hello.c` 到能在终端运行，中间经历了五个阶段，图 2.1 画出了完整链条：

1. **预处理** (`gcc -E`): 把 `#include` 的头文件内容展开进来, 把宏替换成实际文本, 得到 `hello.i`;
2. **编译** (`gcc -S`): 把 C 文本翻译成汇编 (assembly, 一种给人看的助记指令文本), 得到 `hello.s`;
3. **汇编** (`as`): 把汇编指令逐条编码成机器码字节, 得到目标文件 `hello.o`;
4. **链接** (`ld`): 把你的代码和 `printf`、`fgets` 这类库函数的实现拼在一起, 得到可执行文件 `a.out`;
5. **加载执行**: 操作系统把可执行文件读进内存, CPU 从入口地址开始逐条执行机器码。

平时写 `gcc -o hello hello.c` 一条命令搞定, 是因为 `gcc` 在幕后把五步连跑了。

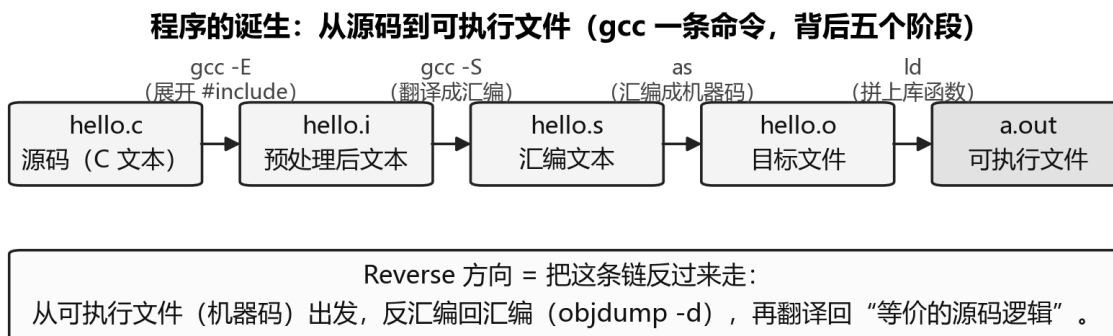


图 2.1: 原理示意图 (matplotlib 绘制): 源码经预处理、编译、汇编、链接变成可执行文件; Reverse 就是把这条链反着走

常见误区

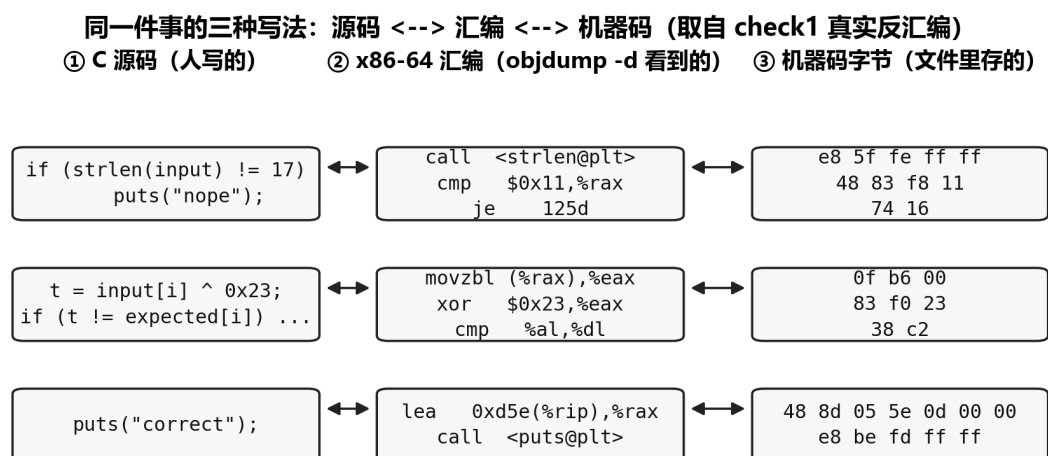
“编译”这个词有两个含义：狭义指 C→汇编那一步，广义指从源码到可执行文件的整个链条。本册说“编译成可执行文件”时指后者。

2.2 源码、汇编、机器码：一件事的三种写法

同一段逻辑在三个层面的写法对照见图 2.2，全部取自本册 R1 题程序 check1 的真实反汇编：

- **源码**：人写的，有变量名、有 if/for；
- **汇编**：CPU 视角的步骤清单。mov 是搬运数据，cmp 是比较并设置标志，je (jump if equal) 是“相等就跳转”，call 是调用函数，xor 是按位异或；
- **机器码**：汇编指令的字节编码。比如 xor \$0x23,%eax 这条指令编码为 3 个字节 83 f0 23。

逆向的过程就是：从文件里读到机器码，用工具还原出汇编（这一步叫反汇编，disassembly），再把汇编“翻译成一句源码的意思”。不需要逐条背指令，入门阶段认识十来条常见指令足够（下一小节给出最小指令集）。



编译器把每条 C 语句翻译成若干条汇编指令，每条汇编指令再编码成几个字节；逆向时我们从 ③ 读出 ②，再把 ② 说成 ① 的意思。

图 2.2: 原理示意图 (matplotlib 绘制)：同一件事的三种写法对照。左：C 源码；中：objdump -d 看到的 x86-64 汇编；右：文件里实际存储的机器码字节

但“认识十来条指令”不等于读得懂一整段反汇编。第 4 章 R1 第 4 步要对着 objdump -d 的输出找变换，输出里还夹着 %eax、%rdi、rbp-0x90 这些没解释过的东西。下面四个小节把这块补齐：寄存器、最小指令集、两种汇编语法、栈帧与调用约定。它们全部绑定到 R1 的真实反汇编，读完就能照着读。

2.2.1 读汇编前的第一课：寄存器是 CPU 手边的小格子

寄存器（register）是 CPU 内部直接参与运算的一小块存储：数量很少（x86-64 有 16 个 64 位的通用寄存器），但速度比内存快得多。程序运行时的中间数据几乎都在寄存器之间倒手，而不是每次读写内存，所以反汇编里全是寄存器的名字。不认识这些“格子”，反汇编就等于看天书。

下表列出入门题里最常出场的几个，第二列标出惯例用途与本册第一次见到它的地方（都来自真实反汇编）：

寄存器	惯例用途与本册出处
%rax	累加器，也是函数返回值的存放处。cmp \$0x11,%rax 比较长度，mov \$0x1,%eax 表示“返回 1、判定失败”
%rcx	第 4 个参数（也常作计数器）。lea 0xd86(%rip),%rcx 把目标数组地址放进 rcx
%rdx	第 3 个参数。mov 0x2e22(%rip),%rdx 把 stdin 交给 fgets
%rsi	第 2 个参数。mov \$0x80,%esi 把缓冲区长度 0x80 交给 fgets
%rdi	第 1 个参数。mov %rax,%rdi 把缓冲区地址交给 fgets
%rbp	栈基址，指向当前函数栈帧的起点。lea -0x90(%rbp),%rax 取局部缓冲区地址
%rsp	栈顶指针，始终指向栈顶。sub \$0xa0,%rsp 给局部变量腾空间
%rip	指令指针，指向“下一条要执行的指令”。lea 0xd86(%rip),%rcx 借此定位常量

还有一条容易卡的规则：同一个寄存器按“用到多少位”有不同的名字。%rax 是 64 位，%eax 是它的低 32 位，%ax 是低 16 位，%al 是最低 8 位（%rdx 同理有 %edx、%dl）。所以 R1 第二段反汇编里出现的 %eax 和 %al，其实是同一个格子的不同宽度：比较长度这类“整型”用 64 位名 %rax，处理单个字节时用 8 位名 %al。

一句话记忆

寄存器是 CPU 手边的小格子：整型参数按 rdi、rsi、rdx、rcx 的顺序传，返回值看 rax，栈顶看 rsp。

常见误区

%eax 不是“另一个寄存器”，而是 %rax 的低 32 位视图。写 mov \$0x1,%eax 会把 rax 的高 32 位一并清零，别把它当成独立变量。

2.2.2 十几条够用的最小指令集

反汇编里只有一条条指令（instruction）。不必逐条背，入门题反复出现的就那么十几条，先认识它们的“一句话含义”：

指令	一句话含义（后附本册真实例子）
mov	搬运数据（寄存器或内存）。mov %rax,%rdi
lea	只算地址、不访问内存（load effective address）。lea -0x90(%rbp),%rax
cmp	比较两个值并设置标志位，不改动原操作数。cmp \$0x11,%rax
test	按位与后设标志，常用来判“是不是 0”。test %rax,%rax
je / jne	相等 / 不相等就跳转。je 1211 <main+0x48>
jmp	无条件跳转。jmp 12d7 <main+0x10e>
call / ret	调用函数 / 从函数返回。call <fgets@plt>
push / pop	把一个值压入 / 弹出栈。push %rbp
xor	按位异或（对同一个寄存器异或自身等于清零）。xor \$0x23,%eax
add / sub	加 / 减。sub \$0xa0,%rsp
and / or	按位与 / 或，常用来取掩码。and \$0xff,%eax
shl / shr	左移 / 右移若干位。shr \$0x4,%eax

一条指令写成“助记符操作数, 操作数”。以 xor \$0x23,%eax 为例：\$0x23 是立即数（immediate，直接写死在指令里的常数，AT&T 语法里带 \$ 前缀），%eax 是参与运算的寄存器。而 movzbl (%rax),%eax 里的 (%rax) 表示“以 rax 里的值当地址，去内存取一个字节，再搬进 eax”。

一句话记忆

先认 mov / lea / cmp / test / je / jne / jmp / call / ret / push / pop / xor / add / sub / and / shl / shr，就够读入门题的反汇编了。

常见误区

读汇编不是“逐条执行、心算到底”，而是回答两个问题：比较了什么、做了什么变换。R1 的两段反汇编正好分别回答“长度是不是 17”和“是不是先异或 0x23 再比较”。

2.2.3 AT&T 与 Intel：同一条指令的两种写法

objdump 默认输出 AT&T 语法，字面像 mov %rsp,%rbp：寄存器带 %、立即数带 \$、内存写成 偏移（基址），而且操作数顺序是“源, 目的”。网上大量教程和 Ghidra/IDA 默认的 Intel

语法规则相反：寄存器不带前缀、内存写成 [基址 + 偏移]，顺序是“目的，源”。两派说的是同一件事，只是书写习惯不同：

AT&T (objdump 默认)	Intel (-M intel)	含义
mov %rsp,%rbp	mov rbp, rsp	把 rsp 的值给 rbp
sub \$0xa0,%rsp	sub rsp, 0xa0	栈顶下移 0xa0 字节
lea -0x90(%rbp),%rax	lea rax, [rbp-0x90]	把 rbp-0x90 这个地址放进 rax
mov \$0x80,%esi	mov esi, 0x80	把 0x80 放进 esi

想切换到 Intel 语法，给 objdump 加一个参数即可：objdump -d -M intel 目标文件 (-M intel 意为“反汇编时用 Intel 语法”)。图 2.3 是真实对照截图：对 check1 的 main 开头分别用两种语法反汇编，机器码字节完全一致，只有写法不同。

```
objdump: AT&T vs Intel syntax

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C objdump -d labs/reverse/check1
| sed -n '/11cd:/,/11fd:/p'
11cd:055          @push    %rbp
11ce:048 89 e5     @mov     %rsp,%rbp
11d1:048 81 ec a0 00 00 00 @sub     $0xa0,%rsp
11d8:064 48 8b 04 25 28 00 @mov     %fs:0x28,%rax
11df:000 00
11e1:048 89 45 f8     @mov     %rax,-0x8(%rbp)
11e5:031 c0          @xor     %eax,%eax
11e7:048 8b 15 22 2e 00 00 @mov     0x2e22(%rip),%rdx      # 4010 <stdin@GLIBC_2.2.5>
11ee:048 8d 85 70 ff ff ff @lea     -0x90(%rbp),%rax
11f5:0be 80 00 00 00 @mov     $0x80,%esi
11fa:048 89 c7       @mov     %rax,%rdi
11fd:0e8 ce fe ff ff @call    10d0 <fgets@plt>

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C objdump -d -M intel
labs/reverse/check1 | sed -n '/11cd:/,/11fd:/p'
11cd:055          @push    rbp
11ce:048 89 e5     @mov     rbp, rsp
11d1:048 81 ec a0 00 00 00 @sub     rsp, 0xa0
11d8:064 48 8b 04 25 28 00 @mov     rax, QWORD PTR fs:0x28
11df:000 00
11e1:048 89 45 f8     @mov     QWORD PTR [rbp-0x8], rax
11e5:031 c0          @xor     eax, eax
11e7:048 8b 15 22 2e 00 00 @mov     rdx, QWORD PTR [rip+0x2e22]      # 4010 <stdin@GLIBC_2.2.5>
11ee:048 8d 85 70 ff ff ff @lea     rax, [rbp-0x90]
11f5:0be 80 00 00 00 @mov     esi, 0x80
11fa:048 89 c7       @mov     rdi, rax
11fd:0e8 ce fe ff ff @call    10d0 <fgets@plt>
```

图 2.3: 真实终端截图：objdump -d（上，AT&T 语法）与 objdump -d -M intel（下，Intel 语法）反汇编 labs/reverse/check1 的 main 开头，机器码相同、写法不同

排错指南

按 Intel 的习惯把 mov %rsp,%rbp 读成“把 rbp 给 rsp”（目的在前）就会把搬运方向搞反，进而把栈的方向想错。看到不熟悉的写法，先判断它属于哪一派，再读顺序。

2.2.4 函数怎么被调用：栈、栈帧与参数传递

R1 第 4 步的反汇编里有连续三行 `lea -0x90(%rbp),%rax`、`mov %rax,%rdi`、`call <fgets@plt>`。要读懂它，得先知道“参数放哪、局部变量放哪”。

栈（stack）是内存里一段“后进先出”的区域，专供函数调用使用。它从高地址向低地址生长：`push` 让栈顶下降、`pop` 让栈顶上升，`%rsp` 始终指向栈顶。一个函数在栈上占用的那一段叫**栈帧**（stack frame），`%rbp` 指向它的起点。

x86-64 Linux 的**调用约定**（calling convention）就三条，记住就能读调用现场：

1. 前 6 个整数/指针**参数**依次放进 `%rdi`、`%rsi`、`%rdx`、`%rcx`、`%r8`、`%r9`，超过 6 个的参数才通过栈传递；
2. 函数的**返回值**放在 `%rax`；
3. 进入函数先 `push %rbp`、`mov %rsp,%rbp` 建立自己的栈帧，再用 `sub $N,%rsp` 给局部变量腾地方；局部变量按 `rbp`-偏移访问。

把这三条套到图 2.3 上半段的真实反汇编，整个场面就立起来了：

- `push %rbp` 加 `mov %rsp,%rbp`：保存调用者的栈基址，建立 main 自己的栈帧；
- `sub $0xa0,%rsp`：给 main 的局部变量开 0xa0（160）字节，源码里那个 `char input[128]` 就在这里；
- 一连串准备动作：`lea -0x90(%rbp),%rax` 取出缓冲区地址；`mov %rax,%rdi` 把它送进第 1 个参数；`mov $0x80,%esi` 送进第 2 个参数（长度 0x80）；`mov 0x2e22(%rip),%rdx` 送进第 3 个参数（stdin）；最后 `call <fgets@plt>` 读出整行；
- `xor %eax,%eax`：把 `eax` 清零（任何数异或自己都是 0），这是编译器准备“返回 0”的惯用手法。

一句话记忆

读函数调用三步曲：看参数进了哪几个寄存器（`rdi/rsi/rdx`...），看 `call` 了谁，看返回值怎么用 `rax`。

常见误区

栈是“从高地址往低地址”长的，所以 `sub $0xa0,%rsp` 是往下开空间，而不是往上。这一点写 Pwn 题构造 payload 时会再次用到（Pwn 分册详解），方向想反会全盘失败。

2.3 二进制里能看到什么：字符串、常量、代码逻辑

可执行文件不是一个黑疙瘩，里面有明确的“分区”。ELF 文件常见分区：

- `.text`: 机器码（程序的逻辑）;
- `.rodata`: 只读数据（read-only data），比如提示语、目标常量数组;
- `.data` / `.bss`: 可写的全局变量;
- `.symtab`: 符号表（symbol table），记录函数名与地址的对应关系。若编译时“去符号”（stripped），函数名就没了，只剩地址，逆向难度随之上升。

所以“二进制里能看到什么”的答案分三层：

1. 可打印字符串：用 `strings` 一搜就有，比如错误提示、菜单文字；
2. 常量：数字数组、密钥、目标值，多藏在 `.rodata`，用 `xxd` 或 `objdump -s` 能直接读出字节；
3. 代码逻辑：藏在 `.text`，需要反汇编阅读。

R1 题的“标准答案数组”就躺在 `.rodata` 里，图 4.4 会带你亲眼看到它。

一句话记忆

字符串看 `strings`，常量看 `.rodata`，逻辑看 `.text` 的反汇编——二进制取证的三层套路。

2.4 逆向的标准路线：黑盒观察 → 白盒验证

“黑盒”指只看输入输出、不看内部；“白盒”指读内部实现。做逆向题的标准姿势是先黑盒、后白盒，最后用原程序验证，全程如图 2.4：

1. 黑盒观察：运行程序，故意喂错误输入，记录它打印什么（nope？哪里报错？）；
2. 静态取证：`file` 看类型、`strings` 找提示、`xxd/objdump -s` 看常量；
3. 找到变换：`objdump -d` 读逻辑，判断输入经过了什么运算（异或？乘加？RC4？）；
4. 写等式反推：把“程序的比较”改写成等式，用 `python3` 解出候选输入；
5. 原程序验证：把候选输入喂回原程序，看到 `correct` 才算数。

逆向题标准路线：黑盒观察 → 静态取证 → 找变换 → 反推 → 用原程序验证

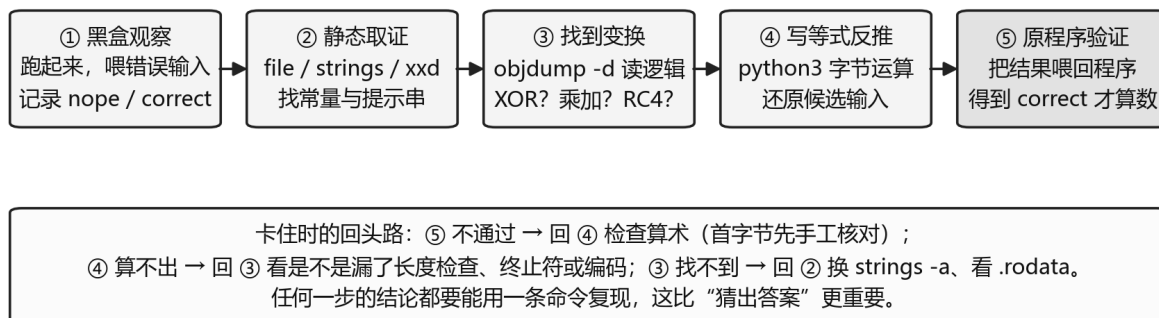


图 2.4: 原理示意图（matplotlib 绘制）：逆向题标准路线与卡住时的回头路

常见误区

脚本打印出一个“像 flag 的字符串”不等于解题完成。只有原程序接受（或平台判定正确），证据链才算闭合。

2.5 字节、ASCII 与十六进制

计算机里没有“字符”，只有数字。**ASCII** 表把数字和字符一一对应：65 是 ‘A’，97 是 ‘a’，102 是 ‘f’，123 是左花括号 ‘{’。所以“把字节 0x66 变成字符”就是查表：chr(0x66) 得到 ‘f’。反过来 ord('f') 得到 102。

十六进制（hex）是看字节的记法：两位数字表示一个字节，前缀 0x 表示这是十六进制数，如 0x23 表示十进制 35。写十六进制的好处是字节边界一目了然——反汇编、xxd 输出全是十六进制，看惯了就会形成“字节直觉”。

python3 是最好的字节计算器，随手可验证：

```

>>> 0x45          # 十六进制 -> 十进制
69
>>> chr(0x66)     # 字节 -> 字符
'f'
>>> ord('f')      # 字符 -> 字节
102
  
```

2.6 异或 (XOR)：自己的逆运算

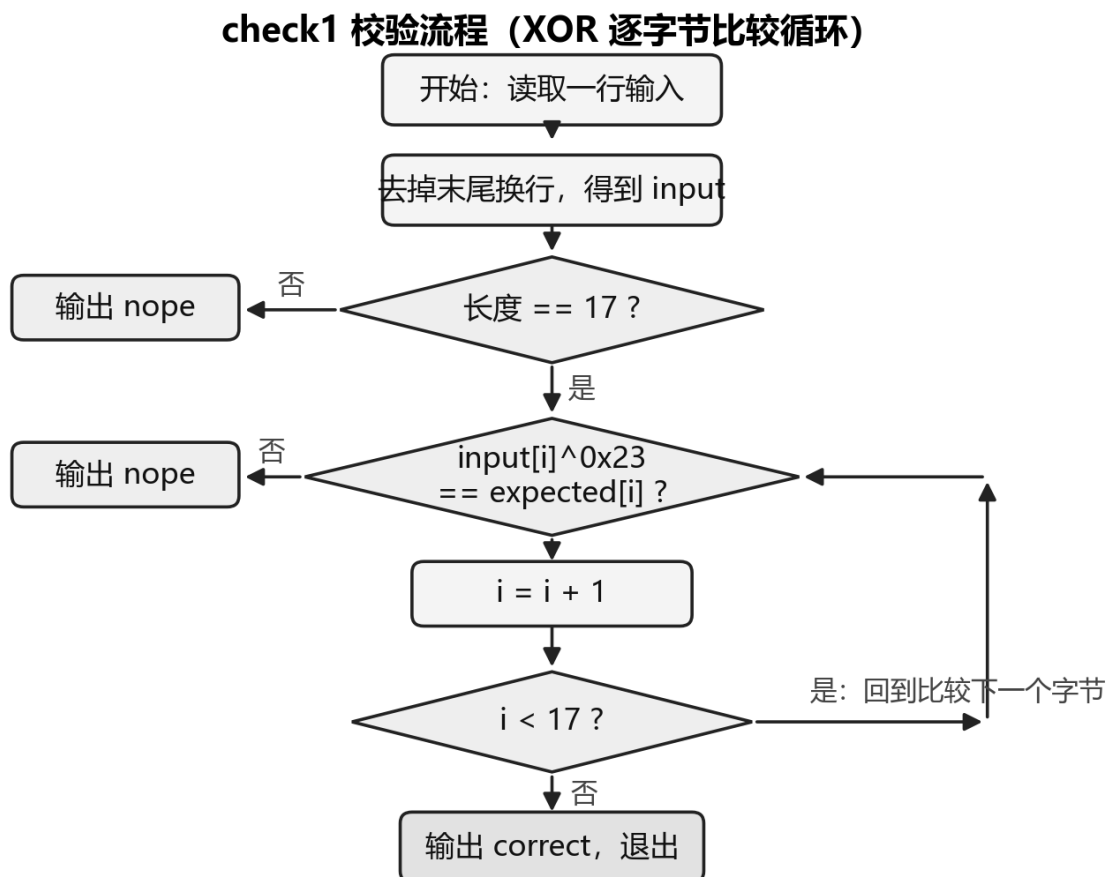
异或 (exclusive or, 写作 $\wedge$ 、EOR、XOR) 是按位运算：两位相同得 0，不同得 1。它有两条逆天的性质：

1. **自己是自己的逆**： $(a \oplus b) \oplus b = a$ 。把数据用密钥 b 异或一次得到密文，再异或一次就还原明文；
2. **任意值异或 0 不变**： $a \oplus 0 = a$ 。

下面的真值表是全部规则：

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

例如 $0x45 \oplus 0x23$ ：先化成二进制 $0100\ 0101 \oplus 0010\ 0011 = 0110\ 0110 = 0x66$ ，也就是字符 ‘f’。R1 的整个校验就是一个“逐字节异或 $0x23$ 再比较”的循环，图 2.5 给出流程图，它的求解就是把把这个循环每一步反过来做一遍异或。



要点: 任一字节不相等就立刻 nope; 17 个字节全部相等才走到 correct。
 求解就是把这条循环反过来: $\text{expected}[i] \wedge 0x23 = \text{input}[i]$ 。

图 2.5: 原理示意图 (matplotlib 绘制): check1 的校验流程。求解时把箭头反过来: 对每个目标字节再异或 0x23 就得到输入

一句话记忆

XOR 加密的“密钥”就是解密的密钥: 同一个运算做两次回到原点。

2.7 Python 字节运算反推: 从 TARGET 逆出 $(b*3+7)\&255$

R2 的程序对每个输入字节 b 计算 $t = (3b + 7) \bmod 256$ 再和目标列表比较。“&255”是程序员写“取模 256”的惯用法: 一个字节只有 8 位, &255 的作用就是丢掉进位、只留低 8 位。

顺带认识两个位运算写法, 读 RC4 代码 (第 2.8 节) 和 557 的推演 (第 7 章) 时都会反复见到:

- **掩码 (mask)**: `0xFF` 就是二进制 `11111111` (8 个 1)。一个数与 `0xFF` 做按位与, 高位的比特全被清成 0, 只保留最低 8 位——所以对非负整数而言 `x & 0xFF`、`x & 255`、`x % 256` 三者等价。RC4 代码里每一处 `& 0xFF` 都是在把索引压回 0–255, 防止越界。
- **移位 (shift)**: `x << n` 把二进制整体左移 `n` 位 (低位补 0), 相当于乘 2^n ; `x >> n` 右移 `n` 位, 相当于整除 2^n 。557 那个自定义 RC4 里的 `(t << 4) | (t >> 4)` 就是把一个字节的 高 4 位和低 4 位对调位置。

要从 t 反推 b , 先想清楚变换是否可逆:

- 加 7 可逆: 减 7 就好;
- 乘 3 在模 256 下可逆, 当且仅当 3 与 256 互素 (最大公约数为 1)。 $3 \times 171 = 513 = 2 \times 256 + 1$, 即 $3 \times 171 \equiv 1 \pmod{256}$, 所以 171 是 3 的**模逆元** (modular inverse);
- 于是 $b = (t - 7) \times 171 \bmod 256$ 。

python3 内置了求模逆的函数: `pow(3, -1, 256)` 直接得到 171, 不用手算。反推代码:

```
TARGET = [57, 75, 42, 60, 120, 99, 93, 42, 48, 54,
           36, 99, 63, 54, 36, 75, 84, 84, 87, 126]
inv3 = pow(3, -1, 256)                # 171
plain = bytes(((t - 7) * inv3) & 255 for t in TARGET)
print(plain.decode())
```

常见误区

如果乘数是 2 (与 256 不互素), 就不存在模逆元: 偶数输出会对应两个可能的输入, 奇数输出根本产生不了。此时不能硬套公式, 要结合字符集、长度等约束缩小范围。判断“先问变换有没有唯一逆”, 是逆向题的分水岭。

2.8 RC4 的两个阶段: KSA 与 PRGA

RC4 是一种经典的**流密码** (stream cipher): 它根据密钥生成一串看起来随机的字节 (叫**密钥流**, keystream), 再把明文和密钥流逐字节异或得到密文。解密是同一过程——用同一个密钥重新生成密钥流, 再异或一次即可还原。

RC4 分两个阶段 (流程见图 2.6):

1. **KSA** (Key Scheduling Algorithm, 密钥调度): 造一张 256 项的状态表 $S = [0, 1, \dots, 255]$, 用密钥把它彻底打乱;
2. **PRGA** (Pseudo-Random Generation Algorithm, 伪随机生成): 每需要一个字节, 就再次搅动状态表并吐出一个密钥流字节 K 。

标准 RC4 的参考实现（与本册 589 解题脚本一致）：

```
def rc4(key, data):
    S = list(range(256))
    j = 0
    for i in range(256):
        # KSA
        j = (j + S[i] + key[i % len(key)]) & 0xFF
        S[i], S[j] = S[j], S[i]
    i = j = 0
    out = bytearray()
    for byte in data:
        # PRGA
        i = (i + 1) & 0xFF
        j = (j + S[i]) & 0xFF
        S[i], S[j] = S[j], S[i]
        out.append(byte ^ S[(S[i] + S[j]) & 0xFF])
    return bytes(out)
```

RC4 的安全性完全依赖密钥保密。攻击 RC4 题目的切入口通常是：找到密钥藏在哪、密文藏在哪，以及程序按什么顺序调用解密。

RC4 的两个阶段：KSA 打乱状态表，PRGA 生成密钥流

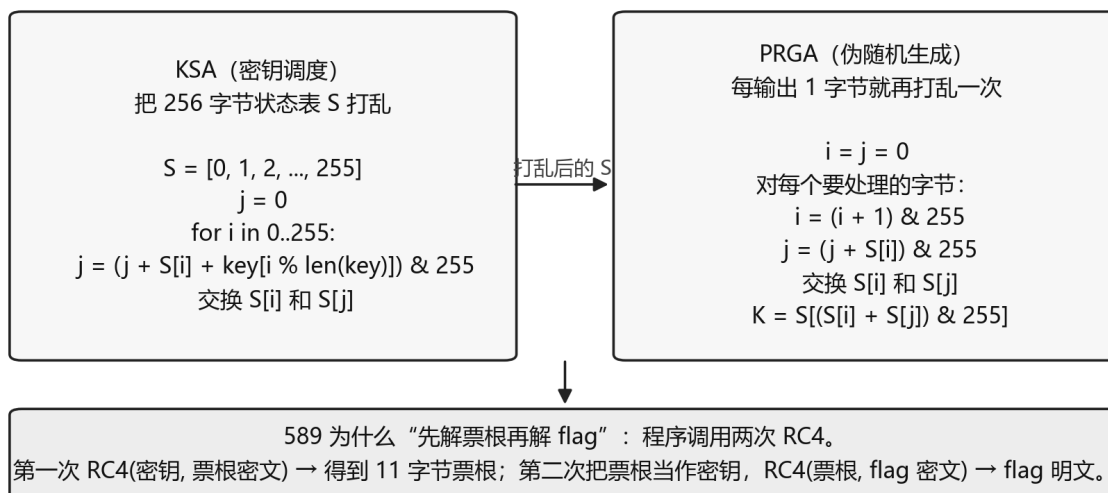


图 2.6: 原理示意图（matplotlib 绘制）：RC4 的 KSA 与 PRGA 两个阶段，以及 589 题“先解票根再解 flag”的两次调用关系

2.8.1 为什么 589 要“先解票根再解 flag”

玄机 589 闸机票根的程序用了两次 RC4：第一次用内置的 16 字节密钥解出 11 字节的票根 (ticket，相当于通关凭证)；第二次把刚解出的票根当作密钥，去解 flag 的密文。两步环环相扣：不知道票根就无法生成第二步的密钥流，直接解 flag 只会得到乱码。这正是流密码“密钥决定一切”的直接体现。第 4 章会用真实脚本把两步跑通。

一句话记忆

RC4 = KSA 打乱状态表 + PRGA 吐密钥流 + 与密文逐字节异或；密钥错了，输出只是乱码，不会报错。

2.9 本章小结：把术语串成一条线

源码被编译器翻译成机器码，存放在可执行文件的 .text；提示语和目标常量放在 .rodata；函数名放在符号表（可能被剥掉）。要读懂 objdump 输出，先认识寄存器与十几条常用指令，以及栈帧与调用约定（第 2.2 节）。拿到题目先黑盒观察建立基线，再用 strings、xxd、objdump 做静态取证，找出输入经历的变换（XOR、乘加、RC4……），把“程序的比较”写成等式反推，最后把结果喂回原程序验证。这条线会在第 4 章走三遍。

第三章 Reverse 工具箱

本章目标：掌握六个入门必备工具：每个都知道“是什么、怎么用、输出怎么读”，并看过真实输出。

本章每个工具都配真实终端截图。所有截图在 WSL 中录制；命令前的长 `cd` 是为了进入资料根目录，你也可以先进入目录再执行短命令。

3.1 file：先问“这是什么”

file 命令读取文件开头的若干字节，告诉你它是什么类型、什么架构、有没有去符号。这是所有分析的第一步，因为不同类型的文件要用不同的工具。

基本用法：

```
file labs/reverse/check1
```

图 3.1 是真实输出，两行都值得逐词看懂：

- **ELF 64-bit LSB pie executable, x86-64:** ELF 是 Linux 可执行文件格式（类似 Windows 的 PE）；64 位、x86-64 架构、小端序（LSB，低位字节存低地址）；
- **not stripped:** 带符号表，函数名可见；**stripped:** 符号被剥离，只剩地址——第二行的 589 附件 `gate_ticket` 就是去符号的，逆向难度更高。

```
file: identify type and stripped symbols

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
file labs/reverse/check1
labs/reverse/check1: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=81dbd74bf545fd811d9406955e4424bcfc1e4f,
for GNU/Linux 3.2.0, not stripped
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
file labs/platform/attachments/589-gate-ticket/gate_ticket
labs/platform/attachments/589-gate-ticket/gate_ticket: ELF 64-bit LSB executable, x86-64, version 1
(SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, stripped
```

图 3.1: 真实终端截图: file 识别 check1 (ELF 64-bit、not stripped) 与 589 附件 gate_ticket (ELF 64-bit、stripped)

一句话记忆

file 三连问: 什么格式 (ELF/PE)? 什么架构 (x86-64)? 有没有符号 (stripped 与否)?

3.2 strings: 翻一翻可读文字

strings 扫描文件中连续的可打印字符序列。它常能直接翻出提示语、错误消息,甚至明文 flag; 但它只能看到**原样存放**的字符串, 异或过的、拆开存的、编码过的都看不见。

基本用法 (-a 表示扫描整个文件, 别只看默认段):

```
strings -a labs/reverse/check1
```

图 3.2 的真实输出里有三类内容: 库函数名 (fgets, strlen……)、一串乱码 (EOBDXQFUFQPF| [LQ~——这其实是目标数组, 后面会讲)、以及两个关键提示 nope 与 correct。没搜到 flag 不能说题目无解, 只能说明它没以连续明文存放。

```
strings: printable strings in check1

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && strings -a
labs/reverse/check1 | head -12
/lib64/ld-linux-x86-64.so.2
mgUa
fgets
stdin
puts
__stack_chk_fail
strlen
strcspn
__libc_start_main
__cxa_finalize
libc.so.6
GLIBC_2.4
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && strings -a
labs/reverse/check1 | tail -10
.fini
.rodata
.eh_frame_hdr
.eh_frame
.init_array
.fini_array
.dynamic
.data
.bss
.comment
```

图 3.2: 真实终端截图: strings -a 翻看 check1。没看到 flag, 但看到 nope / correct 提示和一串“乱码”(目标数组的 ASCII 视图)

3.3 xxd: 用十六进制看字节

xxd 把文件字节以十六进制打印出来, 左边是偏移(地址), 中间是字节, 右边是可读字符视图。看文件头、看常量数组、看“乱码”的真实身份, 都靠它。

基本用法:

```
xxd -l 48 labs/reverse/check1      # 只看前 48 字节
xxd -s 0x2010 -l 32 labs/reverse/check1  # 从偏移 0x2010 看 32 字节
```

图 3.3 的真实输出: 第一段开头的 7f 45 4c 46 就是 ELF 的魔数(magic number, 文件格式的固定签名); 第二段从偏移 0x2010 起是 45 4f 42 44 ..., 右侧 ASCII 视图显示 EOBDXQFUFQPF| [LQ~——正是 R1 的目标数组, 和 strings 看到的“乱码”对上了。

```
xxd: file header and expected[] bytes

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && xxd -l 48 labs/reverse/check1
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF.....
00000010: 0300 3e00 0100 0000 e010 0000 0000 0000  ..>.....
00000020: 4000 0000 0000 0000 9837 0000 0000 0000  @.....7.....
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && xxd -s 0x2010 -l 32 labs/reverse/check1
00002010: 454f 4244 5851 4655 4651 5046 7c5b 4c51  E0BDXQFUFQPF|[LQ
00002020: 5e0a 006e 6f70 6500 636f 7272 6563 7400  ^..nope.correct.
```

图 3.3: 真实终端截图: xxd 看 check1 文件头 (7f 45 4c 46 = ELF 魔数) 与偏移 0x2010 处的目标数组字节

3.4 objdump -d: 反汇编看逻辑

objdump 是 GNU 的二进制分析瑞士军刀, **-d** 参数把机器码反汇编成汇编指令。逆向入门用它的两个姿势:

```
objdump -d labs/reverse/check1 # 全部反汇编
objdump -d labs/reverse/check1 | sed -n '/<main>:/,+14p' # 只看 main 开头
```

`sed -n '/<main>:/,+14p'` 的意思是“从含 `<main>:` 的行开始, 往后打印 14 行”。反汇编输出很长, 用这种方式切片是必备技能。输出的默认语法是 AT&T (如 `%rax`、`$0x11`); 想换成网上更常见的 Intel 语法就加 `-M intel`, 两种写法的区别与读法见第 2.2 节。

图 3.4 的真实输出是 `main` 的开头十几行: `call <fgets@plt>` 表示调用 `fgets` 读输入 (`@plt` 是跳进动态库的中转站), `mov $0x80,%esi` 是给 `fgets` 传缓冲区长度 `0x80` (128 字节)。函数名 (如 `<main>`) 来自符号表; 去符号文件里这些名字都会消失, 只能靠地址。

```
objdump -d: disassemble main

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
objdump -d labs/reverse/check1 | sed -n '/<main>:/,+14p'
0000000000011c9 <main>:
    11c9:0f 0f 1e fa      @endbr64
    11cd:55             @push    %rbp
    11ce:48 89 e5        @mov     %rsp,%rbp
    11d1:48 81 ec a0 00 00 @sub     $0xa0,%rsp
    11d8:64 48 8b 04 25 28 @mov     %fs:0x28,%rax
    11df:00 00
    11e1:48 89 45 f8        @mov     %rax,-0x8(%rbp)
    11e5:31 c0             @xor     %eax,%eax
    11e7:48 8b 15 22 2e 00 @mov     0x2e22(%rip),%rdx      # 4010 <stdin@GLIBC_2.2.5>
    11ee:48 8d 85 70 ff ff @lea     -0x90(%rbp),%rax
    11f5:be 80 00 00 00     @mov     $0x80,%esi
    11fa:48 89 c7          @mov     %rax,%rdi
    11fd:e8 ce fe ff ff @call    10d0 <fgets@plt>
    1202:48 85 c0           @test    %rax,%rax
```

图 3.4: 真实终端截图: objdump -d 反汇编 check1 的 main 开头, 可见 call <fgets@plt> (读输入) 等指令

3.5 readelf -h: 看 ELF 身份证

readelf 读取 ELF 文件的结构信息。-h 显示 **ELF 头** (文件头), 相当于文件的身份证。

基本用法:

```
readelf -h labs/reverse/check1
```

图 3.5 的真实输出中最有用的四行: **Class:** ELF64 (64 位)、**Type:** DYN (位置无关可执行文件, 即 PIE)、**Machine:** Advanced Micro Devices X86-64 (x86-64 架构)、**Entry point address:** 0x10e0 (程序执行的第一条指令地址)。看懂这几行, 就知道该用 32 位还是 64 位的视角读反汇编。(本册截图统一加 LC_ALL=C 环境变量以英文输出为例; 中文系统的输出是中文, 含义相同。)

```
readelf -h: ELF header and architecture

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
readelf -h labs/reverse/check1
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x10e0
  Start of program headers:              64 (bytes into file)
  Start of section headers:              14232 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:                56 (bytes)
  Number of program headers:              13
  Size of section headers:                64 (bytes)
  Number of section headers:              31
  Section header string table index:     30
```

图 3.5: 真实终端截图: readelf -h 查看 check1 的 ELF 头——ELF64、x86-64、入口地址 0x10e0

3.6 python3: 字节运算的计算器

逆向题的“算”几乎都交给 python3。最常用的四类运算:

```
>>> 0x45 ^ 0x23          # 异或
102
>>> chr(102)             # 字节转字符
'f'
>>> pow(3, -1, 256)       # 求模逆元
171
>>> ((97 * 3 + 7) & 255)  # 正向变换 (& 255 = mod 256)
54
```

图 3.6 是这三类运算的真实终端输出: 异或 $0x45 \wedge 0x23$ 得 102 (字符 f)、3 对 256 的模逆元是 171、对 97/98/99 做 $(3b + 7) \& 255$ 得到 [54, 57, 60]。

```
python3: byte math and modular inverse

mzj@Ubuntu-24.04:~$ python3 -c 'print(0x45 ^ 0x23, chr(0x45 ^ 0x23))'
102 f
mzj@Ubuntu-24.04:~$ python3 -c 'print(pow(3, -1, 256))'
171
mzj@Ubuntu-24.04:~$ python3 -c 'print([((b * 3 + 7) & 255) for b in [97, 98, 99]])'
[42, 45, 48]
```

图 3.6: 真实终端截图: python3 做字节运算——异或、求模逆元、正向变换 ((b*3+7)&255)

一句话记忆

解逆向题的“公式三件套”: ^ 异或、& 255 取低 8 位、pow(a,-1,m) 求模逆元。

3.7 本章工具一览

工具	一句话用途	关键参数
file	识别格式/架构/是否去符号	无
strings	找可打印字符串	-a
xxd	十六进制看字节	-l 长度、-s 偏移
objdump	反汇编、看数据段	-d、-s -j 段名
readelf	看 ELF 结构	-h、-S、-s
python3	字节运算与写破解脚本	-c 单行执行

补充两个以后会用到的工具: nm 列出符号表 (函数名与地址); Ghidra/IDA 是图形化反编译工具, 能把汇编还原成伪代码 (伪代码里的变量名是工具推测的, 涉及细节时以汇编和实际运行结果为准)。本册的 WSL 环境没有安装 gdb 调试器, 所有结论用静态证据 + 程序实际运行验证, 这也是真实比赛中常见的情形。

小技巧

把六条命令打成肌肉记忆: file → strings → xxd → objdump -d → readelf -h → python3。前三条建立直觉, 中间两条找证据, 最后一条算答案。

第四章 题目：题面卡与逐题图解带做

本章目标：三道题，每题一节：先用题面卡把“给什么、要什么、考什么”读清楚，紧接着就是这道题的完整解题步骤——读题与思路、分步操作与观察、结果与核对、为什么会成功、排错指南、变式练习。建议先只读题面卡自己动手试 10–15 分钟，卡住再看后面的步骤。

怎么读题面卡：每张题面卡固定七个字段：

- **题目来源：**题目在哪儿，离线能不能做；
- **给定材料：**你手上有什么文件、什么代码；
- **目标：**这题到底要你干什么；
- **交付物：**最后交什么，怎么检查格式；
- **考点：**这题训练的知识点；
- **前置知识：**做之前需要看懂讲义哪几节；
- **难度：**★越多越难。

读卡顺序建议：先看“目标”和“交付物”明确终点，再看“给定材料”盘点资产，最后用“前置知识”查漏补缺。

本章三道题都按同一节奏展开：**读题与思路** → **分步操作与观察** → **结果与核对** → **为什么会成功** → **排错指南** → **变式练习**。截图里的命令都先进入资料根目录；你可以在 WSL 中先执行一次 `cd`，后面就只敲短命令。

4.1 题 R1：程序说 nope，正确输入长什么样？

题 R1 题面卡

题目来源：本地练习，离线可做：labs/reverse/check1（二进制）与 labs/reverse/check1.c（源码）

给定材料：一个 ELF 64 位可执行文件 check1；它从标准输入读一行，判定通过打印 correct，否则打印 nope。资料里同时附了源码 check1.c（白盒练习用）

目标：找出能让 check1 打印 correct 的 17 字节输入

交付物：一个 flag{...} 形式的字符串。核对方式：echo 你的输入 | labs/reverse/check1 输出 correct

考点：逐字节 XOR 校验的逆推；常量数组在.rodata 的定位；黑盒观察与白盒验证的配合

前置知识：2.6 节异或、2.3 节二进制分区、2.2 节汇编与寄存器（第 4 章要用 objdump 读 main）、3.2–3.4 节 strings/xxd/objdump

难度：★☆☆（入门）

读题与思路 看到“程序说 nope”，先别急着读代码，做三十秒黑盒观察：它到底怎么失败的？是不管输什么都 nope，还是不同的输入有不同的反应？题面说输入会被逐字节异或 0x23 后和目标数组比较——如果先看到了这句话，思路就是一句话：**对目标数组每个字节再异或 0x23**。但做题时你未必有这句话，所以本节按“先黑盒、后白盒”的完整顺序走一遍：先运行看反应，再看源码与汇编找变换，最后写脚本反推并验证。反推时先手工算第一个字节建立信心： $0x45 \oplus 0x23 = 0x66$ ，查 ASCII 表是 ‘f’——像 flag 的开头，方向对了。

分步操作与观察

1. 确认程序身份。在 WSL 执行：

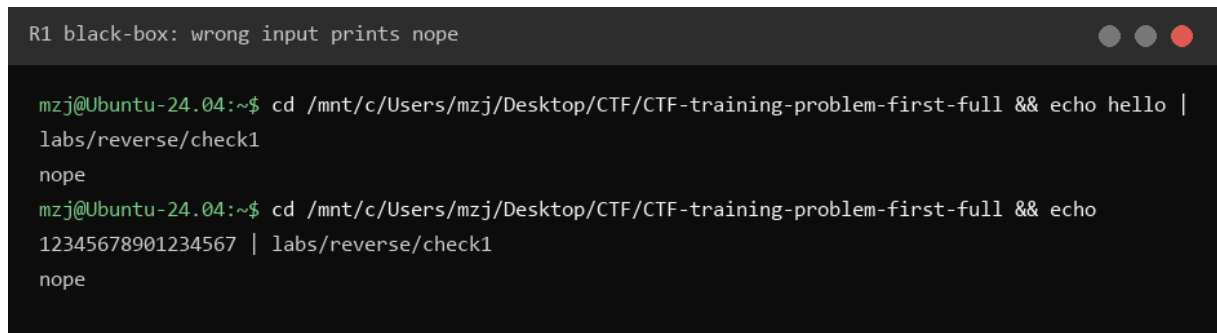
```
cd /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包
LC_ALL=C file labs/reverse/check1
```

这一步是为了确认它是能直接运行的 Linux ELF。应该看到 ELF 64-bit LSB pie executable, x86-64（见图 3.1）。如果显示 PE 或别的格式，后面的命令就要换工具。

2. 黑盒观察：喂两类错误输入。

```
echo hello | labs/reverse/check1
echo 12345678901234567 | labs/reverse/check1
```

第一条是长度错误的输入（5 字节），第二条是长度正确（17 字节）但内容错误的输入。图 4.1 的真实输出：两条都是 nope。这一步建立基线——失败时只会说 nope；也提醒我们：长度和内容是两道关卡，接下来要分别确认。



```
R1 black-box: wrong input prints nope

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && echo hello | labs/reverse/check1
nope

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && echo 12345678901234567 | labs/reverse/check1
nope
```

图 4.1: 真实终端截图: 喂长度错误 (hello) 与长度正确但内容错误 (12345678901234567) 的输入, 都得到 nope

3. 读源码, 把校验写成四句话 (白盒练习可以直接看资料里的 `check1.c`):

```
cat labs/reverse/check1.c
```

图 4.2 的真实输出就是源码全文。把逻辑提炼成四句: 读一行; 去掉末尾换行; 长度必须是 17 (`sizeof expected`); 每个字节异或 0x23 后必须等于 `expected[i]`。提炼高层描述比逐行抄代码管用。

```
R1 white-box: read check1.c

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && cat
labs/reverse/check1.c
/* Exercise R1: find an input accepted by the program. */
static const unsigned char expected[] = {
    0x45, 0x4f, 0x42, 0x44, 0x58, 0x51, 0x46, 0x55, 0x46, 0x51,
    0x50, 0x46, 0x7c, 0x5b, 0x4c, 0x51, 0x5e
};
int main(void) {
    char input[128];
    if (!fgets(input, sizeof input, stdin)) return 1;
    input[strcspn(input, "\n")] = '\0';
    if (strlen(input) != sizeof expected) {
        puts("nope");
        return 1;
    }
    for (size_t i = 0; i < sizeof expected; i++) {
        if (((unsigned char)input[i] ^ 0x23) != expected[i]) {
            puts("nope");
            return 1;
        }
    }
    puts("correct");
    return 0;
}
```

图 4.2: 真实终端截图: cat 查看 check1.c 源码——目标数组、长度检查与异或比较循环

4. (模拟无源码) 用反汇编找证据。真实比赛通常没有源码, 用 objdump 看 main 的两段关键逻辑:

```
LC_ALL=C objdump -d labs/reverse/check1 | sed -n '/1232:/,/125b:/p'
LC_ALL=C objdump -d labs/reverse/check1 | sed -n '/126a:/,/12b1:/p'
```

第一段能看到 `call <strlen@plt>` (求长度) 和 `cmp $0x11,%rax` (和 0x11=17 比较); 第二段能看到 `xor $0x23,%eax` (异或 0x23) 和 `cmp %a1,%d1` (逐字节比较), 见图 4.3。汇编里的立即数 (0x11、0x23) 就是变换的全部秘密。

```

R1 asm evidence: length check + XOR loop

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
objdump -d labs/reverse/check1 | sed -n '/1232:/,/125b:/p'
1232:48 8d 85 70 ff ff ff lea    -0x90(%rbp),%rax
1239:48 89 c7             mov    %rax,%rdi
123c:e8 5f fe ff ff     call   10a0 <strlen@plt>
1241:48 83 f8 11         cmp    $0x11,%rax
1245:74 16              je     125d <main+0x94>
1247:48 8d 05 d5 0d 00 00 lea    0xdd5(%rip),%rax    # 2023 <expected+0x13>
124e:48 89 c7             mov    %rax,%rdi
1251:e8 3a fe ff ff     call   1090 <puts@plt>
1256:b8 01 00 00 00     mov    $0x1,%eax
125b:eb 7a              jmp    12d7 <main+0x10e>

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
objdump -d labs/reverse/check1 | sed -n '/126a:/,/12b1:/p'
126a:48 8d 95 70 ff ff ff lea    -0x90(%rbp),%rdx
1271:48 8b 85 68 ff ff ff mov    -0x98(%rbp),%rax
1278:48 01 d0             add    %rdx,%rax
127b:0f b6 00             movzbl (%rax),%eax
127e:83 f0 23             xor    $0x23,%eax
1281:89 c2              mov    %eax,%edx
1283:48 8d 0d 86 0d 00 00 lea    0xd86(%rip),%rcx    # 2010 <expected>
128a:48 8b 85 68 ff ff ff mov    -0x98(%rbp),%rax
1291:48 01 c8             add    %rcx,%rax
1294:0f b6 00             movzbl (%rax),%eax
1297:38 c2              cmp    %al,%dl
1299:74 16              je     12b1 <main+0xe8>
129b:48 8d 05 81 0d 00 00 lea    0xd81(%rip),%rax    # 2023 <expected+0x13>
12a2:48 89 c7             mov    %rax,%rdi
12a5:e8 e6 fd ff ff     call   1090 <puts@plt>
12aa:b8 01 00 00 00     mov    $0x1,%eax
12af:eb 26              jmp    12d7 <main+0x10e>
12b1:48 83 85 68 ff ff ff addq    $0x1,-0x98(%rbp)

```

图 4.3: 真实终端截图: objdump -d 关键片段。上: 长度检查 (cmp \$0x11); 下: XOR 0x23 的比较循环 (xor \$0x23,%eax)

5. 定位目标数组。汇编里 lea 0xd86(%rip),%rcx 指向符号 <expected>, 说明比较的另一侧是名为 expected 的常量数组。用 objdump 看数据段:

```
LC_ALL=C objdump -s -j .rodata labs/reverse/check1
```

图 4.4 的真实输出里, 偏移 2010 处一行 454f4244 58514655 46515046 7c5b4c51 加下一行开头的 5e, 共 17 字节, 右侧 ASCII 视图显示 E0BDXQFUFQPF| [LQ^——和 strings 看到的“乱码”一致。这就是目标数组, 一共 17 项, 也印证了长度检查。

```
R1 data evidence: expected[] in .rodata

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
objdump -s -j .rodata labs/reverse/check1
labs/reverse/check1:      file format elf64-x86-64
Contents of section .rodata:
 2000 01000200 00000000 00000000 00000000 .....
 2010 454f4244 58514655 46515046 7c5b4c51 EOBDXQFUFQPF|[LQ
 2020 5e0a006e 6f706500 636f7272 65637400 ^..nope.correct.
```

图 4.4: 真实终端截图: `objdump -s -j .rodata` 显示目标数组字节 (45 4f 42 44 ... 5e) 与 ASCII 视图 EOBDXQFUFQPF|[LQ^

6. 写破解脚本反推。对每个目标字节再异或 0x23:

```
expected = [0x45, 0x4f, 0x42, 0x44, 0x58, 0x51, 0x46, 0x55,
            0x46, 0x51, 0x50, 0x46, 0x7c, 0x5b, 0x4c, 0x51, 0x5e]
plain = bytes(x ^ 0x23 for x in expected)
print(plain.decode())
```

截图中用单行命令真实执行了同样的运算:

```
python3 -c 'E=[0x45,0x4f,0x42,0x44,0x58,0x51,0x46,0x55,0x46,0x51,0x50,0x46,0x7c,0x5b,0x4c,0x51,0x5e]; print(bytes(x ^ 0x23 for x in E).decode())'
```

图 4.5 的真实输出: `flag{reverse_xor}`。17 个字符, 长度刚好对上。

```
R1 solver: XOR 0x23 each byte

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && python3 -c
'E=[0x45,0x4f,0x42,0x44,0x58,0x51,0x46,0x55,0x46,0x51,0x50,0x46,0x7c,0x5b,0x4c,0x51,0x5e];
print(bytes(x ^ 0x23 for x in E).decode())'
flag{reverse_xor}
```

图 4.5: 真实终端截图: `python3` 破解脚本真实输出——逐字节 XOR 0x23 得到 `flag{reverse_xor}`

7. 把结果喂回原程序。

```
echo flag{reverse_xor} | labs/reverse/check1
```

图 4.6 的真实输出: `correct`。至此证据链闭环: 黑盒基线、白盒变换、脚本反推、原程序接受, 四样齐全。

```
R1 verify: correct input prints correct

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && echo
flag{reverse_xor} | labs/reverse/check1
correct
```

图 4.6: 真实终端截图：正确输入喂回原程序，输出 `correct`

结果与核对 正确输入（即本题 flag）：

```
flag: flag{reverse_xor}
```

独立核对有两条路：①把它逐字节再异或 0x23，应还原出目标数组的 17 个字节；②喂给原程序，看到 `correct`。两条都过才算验证完成，只跑破解脚本打印字符串还不够。

为什么会成功 一句话机制：异或是自己的逆运算。程序的判定是 $input[i] \oplus 0x23 = expected[i]$ ，等式两边同时异或 0x23，得到 $input[i] = expected[i] \oplus 0x23$ 。展开说：异或不丢失信息，任何一个字节和 0x23 异或后都唯一对应另一个字节；所以“加密”用的 0x23 就是“解密”用的 0x23。目标数组放在 `.rodata` 里没有被加密，这让攻击者能直接读到比较对象——真实恶意软件通常会目标值藏得更深（比如再编码一层）。

排错指南

1. 提示 `nope` 但脚本输出的字符串看起来对：先数长度。echo 会补一个换行符，程序会去掉行尾换行，但如果输入里多了空格（复制时带上行尾空格），长度就不是 17。用 `echo -n` 或管道时留意不可见字符。
2. 输出乱码而不是 flag：检查是否对每个字节都异或了 0x23，且没有漏掉字节数组的逗号——python 里少写一项，后续全部错位。
3. 反汇编里找不到 `xor $0x23`：确认你反汇编的是同一个文件（可用 `md5sum labs/reverse/check1` 记录哈希），或编译参数不同导致代码形状不同——但常量 0x23 和 0x11 一般仍在。
4. echo 管道运行后没有任何输出：确认命令里的路径正确，且在 WSL 里执行（Windows 下无法直接运行 ELF）。

变式练习

1. 把 `check1.c` 复制出来，把 0x23 改成 0x37 重新编译，用同样的方法求新输入；体会“换密钥不换结构”。
2. 把比较方式从“逐字节异或”改成“先异或 0x23 再加 1”，写出新的反推公式，并验证。

3. 只用二进制（把 `check1.c` 藏起来）：从 `objdump -s -j .rodata` 的输出直接抄目标数组，走一遍无源码流程。

4.2 题 R2：乘 3 加 7 的循环，怎么倒着走？

题 R2 题面卡

题目来源：本地练习，离线可做：labs/reverse/verify2.py（Python 验证器）

给定材料：一个 Python 脚本，读取一行输入，对每个字节做变换后与目标列表比较，全对打印 `correct`，否则 `nope`。目标列表写死在脚本里

目标：还原出验证器接受的 20 字节输入

交付物：一个 `flag{...}` 形式的字符串。核对方式：`echo 你的输入 | python3 labs/reverse/verify2.py` 输出 `correct`

考点：模 256 乘加变换的可逆性判断；模逆元的使用；从目标值反推输入的完整流程

前置知识：2.7 节字节运算反推、3.6 节 python3、附录 C 术语“模逆元”

难度：★★☆（基础）

读题与思路 R2 是 Python 写的验证器，读题后先问一个关键问题：这个变换有没有唯一逆？题面给出变换 $t = (3b + 7) \bmod 256$ 。加法可逆（减 7），乘法在模 256 下要 3 与 256 互素才有逆元。 $3 \times 171 = 513 = 2 \times 256 + 1$ ，所以逆元是 171，每个目标值 t 对应唯一的 $b = (t - 7) \times 171 \bmod 256$ 。思路成立后，动手顺序：先黑盒看反应，再读脚本抄下变换和目标列表，然后写反推脚本，最后验证。

分步操作与观察

1. 黑盒观察。随便输一个答案，再输一个“看起来像 flag”的答案：

```
echo hello | python3 labs/reverse/verify2.py
echo flag{reverse_xor} | python3 labs/reverse/verify2.py
```

图 4.7 的真实输出：两条都是 `input> nope`（`input>` 是脚本的提示符）。R1 的答案在这里不通过，说明两题算法不同，不能套用。

```
R2 black-box: wrong input prints nope

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && echo hello |
python3 labs/reverse/verify2.py
input> nope
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && echo
flag{reverse_xor} | python3 labs/reverse/verify2.py
input> nope
```

图 4.7: 真实终端截图: 随便输入与沿用 R1 答案都得到 nope

2. 读脚本, 抄下变换与目标。

```
cat labs/reverse/verify2.py
```

图 4.8 的真实输出是脚本全文。逐行读关键句: `data = candidate.encode("ascii", errors="ignore")` 先把输入转成字节; `((b * 3 + 7) & 255) for b in data` 就是变换 $t = (3b + 7) \bmod 256$; `== TARGET` 是逐项比较。 `len(data) == len(TARGET)` 说明答案长度是 20。

```
R2 white-box: read verify2.py

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && cat
labs/reverse/verify2.py
"""Exercise R2: derive the input accepted by this validator."""
TARGET = [57, 75, 42, 60, 120, 99, 93, 42, 48, 54, 36, 99, 63, 54, 36, 75, 84, 84, 87, 126]
def check(candidate: str) -> bool:
    data = candidate.encode("ascii", errors="ignore")
    return len(data) == len(TARGET) and [((b * 3 + 7) & 255) for b in data] == TARGET
if __name__ == "__main__":
    answer = input("input> ")
    print("correct" if check(answer) else "nope")
```

图 4.8: 真实终端截图: cat 查看 verify2.py——目标列表 TARGET 与变换 $(b*3+7)\&255$

3. 先确认逆元, 再反推。数学先行, 代码在后:

```
python3 -c 'print(pow(3, -1, 256))'
python3 -c 'T=[57,75,42,60,120,99,93,42,48,54,36,99,63,54,36,75,84,84,87,126];
    print(bytes(((t - 7) * 171) & 255 for t in T).decode())'
```

第一条命令输出 171, 确认逆元; 第二条对 20 个目标值逐项反推并按 ASCII 解码。图 4.9 的真实输出: `flag{trace_the_loop}`。

```

R2 solver: mod-inverse 171 then decode

mzj@Ubuntu-24.04:~$ python3 -c 'print(pow(3, -1, 256))'
171
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && python3 -c
'T=[57,75,42,60,120,99,93,42,48,54,36,99,63,54,36,75,84,84,87,126]; print(bytes(((t - 7) * 171) &
255 for t in T).decode())'
flag{trace_the_loop}

```

图 4.9: 真实终端截图: python3 求出模逆元 171, 再逐项还原目标列表, 输出 flag{trace_the_loop}

4. 双通道验证。把结果喂回验证器, 同时反向复算一次:

```

echo flag{trace_the_loop} | python3 labs/reverse/verify2.py
python3 -c 'print([
    ((ord(c)*3+7)&255) for c in "flag{trace_the_loop}"
])'

```

第一条得到 input> correct; 第二条把答案的 ASCII 码正向变换, 输出与 TARGET 一致, 见图 4.10。反推、正算、原程序三者互相印证。

```

R2 verify: correct + forward re-check

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && echo
flag{trace_the_loop} | python3 labs/reverse/verify2.py
input> correct
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && python3 -c
'P=[102,108,97,103,123,116,114,97,99,101,95,116,104,101,95,108,111,111,112,125]; print([(b * 3 +
7) & 255) for b in P])'
[57, 75, 42, 60, 120, 99, 93, 42, 48, 54, 36, 99, 63, 54, 36, 75, 84, 84, 87, 126]

```

图 4.10: 真实终端截图: 验证器输出 correct; 正向复算结果与目标列表完全一致

结果与核对 正确输入 (即本题 flag):

```
flag: flag{trace_the_loop}
```

核对: ① echo flag{trace_the_loop} | python3 labs/reverse/verify2.py 输出 correct; ②正向复算 $(3b + 7) \& 255$ 的 20 个结果逐项等于 TARGET; ③首字节手工核对: $(57 - 7) \times 171 \bmod 256 = 102$, ASCII 是 'f'。

为什么会成功 一句话机制：乘 3 在模 256 下有唯一的逆元 171。展开说：3 与 256 互素，保证了每个输出值都恰好来自一个输入值，变换是“双射”（一一对应）；因此反推公式 $b = (t - 7) \times 171 \bmod 256$ 对全部 20 个字节同时成立。脚本先 `encode("ascii")` 再运算，说明答案是纯 ASCII 文本——这也是我们可以用 `.decode()` 直接读出 flag 的原因。

排错指南

1. 反推结果是乱码：先验证逆元——执行 `pow(3, -1, 256)` 必须得 171；若你把公式写成 $(t - 7)/3$ 或用了 171 的近似值，结果全错。
2. 前几个字符像 flag 后面乱掉：多半是目标列表抄漏或抄错序。数一数列表必须是 20 项，和 `len(TARGET)` 一致。
3. 验证器一直 nope：检查是否把反推结果的行尾换行也算进长度（脚本的 `input()` 会自动去掉换行，一般没问题）；再检查是否多复制了空格。
4. 想改乘数做变式却算不出逆元：先算 `pow(5, -1, 256)` 之类确认存在；乘数取偶数时逆元不存在，这是数学限制，不是你算错。

变式练习

1. 把乘数改成 5（`pow(5, -1, 256)` 存在），重新生成 TARGET 并反推，体会“换参数不换方法”。
2. 把乘数改成 2，观察哪些目标值反推出两个候选字节，思考为什么此时不能只靠公式。
3. 给变换再加一步“结果循环左移 1 位”，写出新的正逆公式，并用双通道验证法自证。

4.3 题 R2+：玄机 589 闸机票根——票根是什么，flag 又藏在哪？

题 R2+ 题面卡

题目来源：玄机平台 2026 安网杯“闸机票根”（challenge 589，<https://xj.edisec.net/challenges/589>）；附件已随资料保存在 labs/platform/attachments/589-gate-ticket/，离线可解

给定材料：一个去符号的 ELF 64 位可执行文件 gate_ticket。运行它会提示 GATE TICKET：等你输入票根；输错有三种不同的拒绝信息，输对则打印 GATE OPEN

目标：先还原 11 字节票根，再用票根解出 flag

交付物：票根字符串与一个 flag{...} 字符串；用随资料提供的脚本 labs/platform/solve_589_gate_ticket.py 可直接复现两行输出

考点：RC4 流密码的两次调用（KSA/PRGA）；从.rodata 提取密钥与密文；“先解票根再解 flag”的依赖关系

前置知识：2.2 节汇编与寄存器、2.8 节 RC4、3.4-3.5 节 objdump/readelf、第 4 章 R1 的带做经验

难度：★★★（进阶）

常见误区

题面卡中 589 的平台信息整理自题目文字与本地附件分析，示意，非平台截图。本地脚本输出的 ticket 与 flag 是真实运行结果（见图 4.13）。

读题与思路 这是一道平台题的本地附件版。程序运行后提示 GATE TICKET：，像门禁一样等你刷票根。先黑盒试一下就知道：随便输会得到三种拒绝信息之一（长度不对、字符集不对、校验失败），说明它对票根有格式和内容的双重检查。题面卡说这题考 RC4 两次调用——先解票根，再用票根当密钥解 flag。所以思路是三步：从二进制里找到密钥与两段密文 → 用 RC4 解出票根 → 用票根解出 flag。这题比 R1/R2 多了两个难点：程序去符号（函数名不可见），常量混在.rodata 里需要定位。好在随资料提供了完整解题脚本，我们先看懂它再运行它。

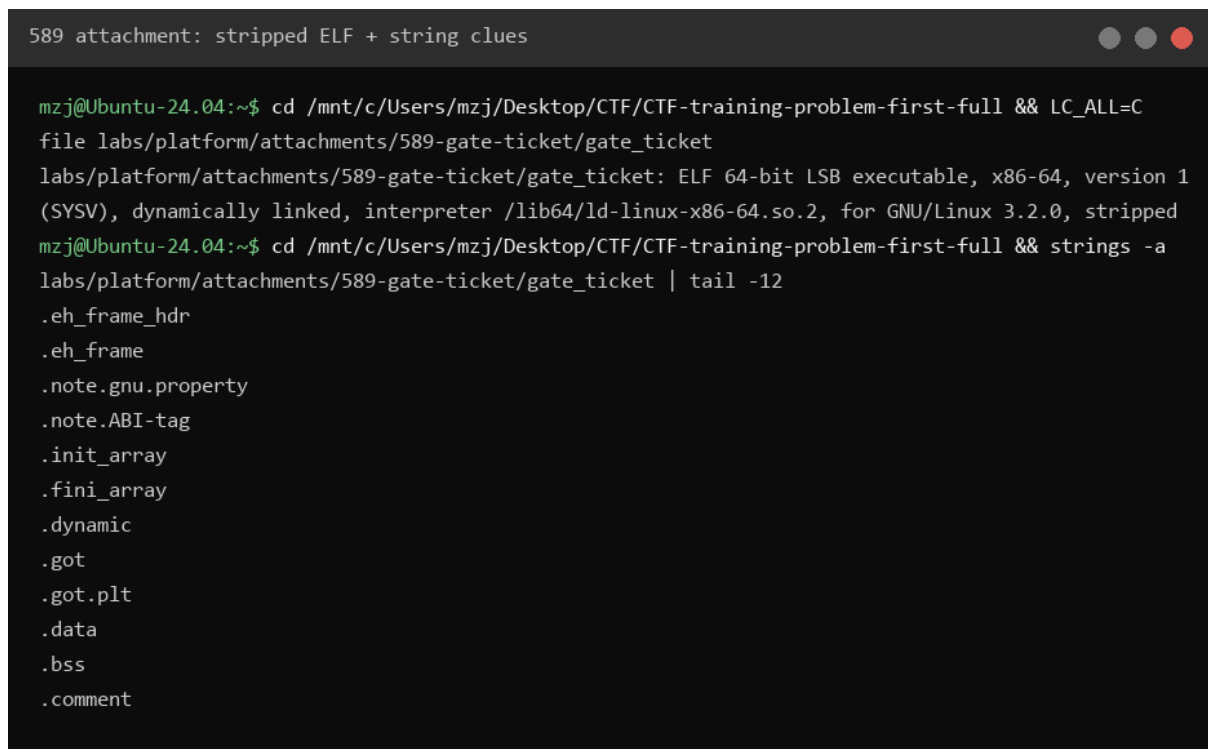
分步操作与观察

1. 识别附件。

```
LC_ALL=C file labs/platform/attachments/589-gate-ticket/gate_ticket
strings -a labs/platform/attachments/589-gate-ticket/gate_ticket | tail -12
```

图 4.11 的真实输出：file 显示 ELF 64-bit LSB executable, x86-64, ..., stripped——注意 stripped，符号表被剥离，反汇编里不会有函数名。strings 的尾部翻出六条有

价值的线索：提示语 GATE TICKET：、三种拒绝信息、GATE OPEN，以及神秘的四字符标记 GTK1。这些字符串是后续定位数据的“路标”。



```
589 attachment: stripped ELF + string clues

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
file labs/platform/attachments/589-gate-ticket/gate_ticket
labs/platform/attachments/589-gate-ticket/gate_ticket: ELF 64-bit LSB executable, x86-64, version 1
(SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, stripped
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && strings -a
labs/platform/attachments/589-gate-ticket/gate_ticket | tail -12
.eh_frame_hdr
.eh_frame
.note.gnu.property
.note.ABI-tag
.init_array
.fini_array
.dynamic
.got
.got.plt
.data
.bss
.comment
```

图 4.11: 真实终端截图：file 识别 gate_ticket (stripped, 去符号)；strings 尾部显示提示语、拒绝信息与 GTK1 标记

2. 看.rodata，按 GTK1 标记定位常量。数据段是这类题的藏宝图：

```
LC_ALL=C objdump -s -j .rodata labs/platform/attachments/589-gate-ticket/
gate_ticket
```

图 4.12 的真实输出里，地址 402080 处有 38 字节看不懂的字节（7b860a12 c bdf7761 ...），地址 4020c0 处是 GTK1 标记，紧随其后是 16 字节 0be84416 6ce2d272 4fc2bc99 5bb6cfa8（在标记偏移 +5 处）和 11 字节 ac67fa6a 32753626 777a536c（标记偏移 +21 处）。对照解题脚本的注释：16 字节那段是 **密钥**，11 字节那段是 **票根密文**，而 402080 的 38 字节是 **flag 密文**（38 字节恰好是 flag{...} 32 位十六进制摘要的长度）。

```

589 .rodata: GTK1 marker, key, ciphertexts

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && LC_ALL=C
objdump -s -j .rodata labs/platform/attachments/589-gate-ticket/gate_ticket
labs/platform/attachments/589-gate-ticket/gate_ticket:      file format elf64-x86-64
Contents of section .rodata:
 402000 01000200 47415445 20544943 4b45543a ....GATE TICKET:
 402010 20000d0a 00746963 6b657420 6c656e67 ....ticket leng
 402020 74682072 656a6563 7465640a 00746963 th rejected..tic
 402030 6b657420 63686172 73657420 72656a65 ket charset reje
 402040 63746564 0a007469 636b6574 20636865 cted..ticket che
 402050 636b2066 61696c65 640a0047 41544520 ck failed..GATE
 402060 4f50454e 0a000000 00000000 00000000 OPEN.....
 402070 00000000 00000000 00000000 00000000 .....
 402080 7b860a12 cbdf7761 9a51fb88 b11c4bdf {...wa.Q...K.
 402090 568746a1 bedda407 1630c83d ad8ff1b0 V.F.....0.=...
 4020a0 715e51d0 9a300000 00000000 00000000 q^Q..0.....
 4020b0 00000000 00000000 00000000 00000000 .....
 4020c0 47544b31 0be84416 6ce2d272 4fc2bc99 GTK1..D.l..r0...
 4020d0 5bb6cfa8 ac67fa6a 32753626 777a536c [...g.j2u6&wzS1

```

图 4.12: 真实终端截图: gate_ticket 的.rodata——402080 处 38 字节 flag 密文、4020c0 处 GTK1 标记及其后的 16 字节密钥与 11 字节票根密文

3. 读懂解题脚本。脚本 labs/platform/solve_589_gate_ticket.py 做了四件事（全文 77 行，值得通读）：

- (a) 用 python 直接解析 ELF 结构、抽出 .rodata 段字节（不依赖 Linux 环境，也不运行陌生程序）；
- (b) 以 GTK1 标记为基准切出密钥（16 字节）与票根密文（11 字节），并按反汇编注释定位 flag 密文（38 字节）；
- (c) 调用标准 RC4（KSA + PRGA，见 2.8 节）解票根，并校验票根是 11 字节可打印字符、往返异或自洽；
- (d) 用票根作密钥解 flag 密文，打印两行结果。

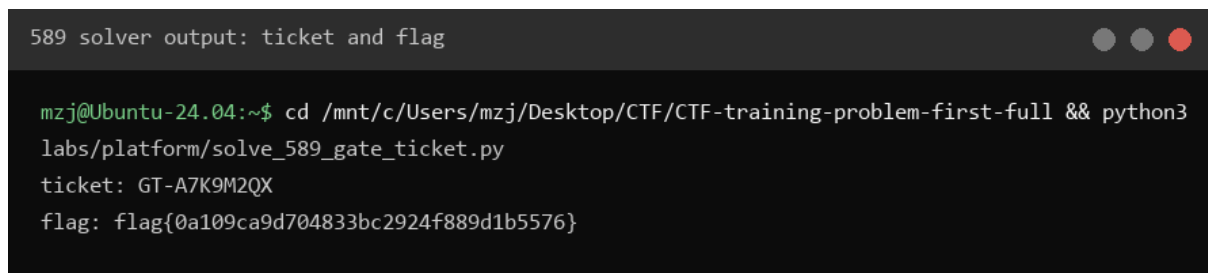
4. 真实运行。

```
python3 labs/platform/solve_589_gate_ticket.py
```

图 4.13 的真实输出：

```
ticket: GT-A7K9M2QX
flag: flag{0a109ca9d704833bc2924f889d1b5576}
```

两行输出对应两次 RC4 调用：第一行是“密钥解票根”，第二行是“票根解 flag”。



```
589 solver output: ticket and flag

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && python3
labs/platform/solve_589_gate_ticket.py
ticket: GT-A7K9M2QX
flag: flag{0a109ca9d704833bc2924f889d1b5576}
```

图 4.13: 真实终端截图: solve_589_gate_ticket.py 真实运行输出——ticket: GT-A7K9M2QX 与 flag（本册所有 flag 中唯一的平台题答案，由真实运行得到）

5.（可选）上机刷票根。如果想看程序亲口说 GATE OPEN，把票根喂给它：

```
echo GT-A7K9M2QX | labs/platform/attachments/589-gate-ticket/gate_ticket
```

这是对“票根正确性”的第四重验证。

结果与核对 票根与 flag（均由脚本真实运行得到）：

```
flag: GT-A7K9M2QX
```

```
flag: flag{0a109ca9d704833bc2924f889d1b5576}
```

独立核对的三条路：①脚本内置的往返校验（用密钥加密票根应还原票根密文）；②把票根喂给 gate_ticket 得到 GATE OPEN；③ flag 形如 32 位十六进制摘要包裹 flag{...}，与 38 字节密文长度吻合。平台提交记录见附录 A（589 已在平台提交并得到正确反馈）。

为什么会成功 一句话机制：RC4 解密就是再执行一遍相同的密钥流生成，两次调用的密钥分别是“内置密钥”和“解出的票根”。展开说：KSA 用密钥打乱 256 项状态表，PRGA 吐出与明文等长的密钥流，逐字节异或即得密文；因为 RC4 是对称的，同一个密钥再跑一遍就还原明文。程序把票根当第二步的密钥，构成“连锁锁”——这也解释了为什么必须先解票根：没有 11 字节的票根，第二步的 KSA 无从开始，直接解 flag 只会得到乱码。去符号只增加了“找函数名”的难度，不改变数据和算法本身。

排错指南

1. 脚本报 `GTK1 data marker was not found`: 说明你用的附件不是资料包里的那一份。用 `md5sum` 或 `SHA-256` 核对附件版本, 平台换过附件就要重新分析。

2. 报 `decrypted ticket is not the expected printable 11-byte value`: 多半是密钥/密文切片偏移搞错 (标记 +5 与 +21 的位置)。对照 图 4.12 逐字节检查。

3. 输出是乱码: 确认用的是标准 RC4 (KSA 里 $j = (j + S[i] + \text{key}[i \% \text{len}(\text{key})]) \& 0xFF$), 没有漏掉 `swap` 或把 PRGA 的 `i/j` 更新顺序弄反。

4. 想用现成 RC4 库却结果不对: 库的接口可能要求明文密钥类型不同 (`str/bytes`); 本题密钥是 16 原始字节, 不是十六进制文本, 别做多余的编解码。

变式练习

1. 改动脚本: 先只解票根不解 flag, 确认第一行输出稳定后再加上第二步, 体会“分步验证”。
2. 把 flag 密文和票根密文在脚本里对调, 观察输出变成什么, 理解“密钥/密文角色由调用顺序决定”。
3. 对照原讲义的 557 qgd 题 (第 7 章保留推演): 它的解密是 128 项状态表的自定义 RC4 变种, 找出与本题标准 RC4 的三个不同点。

4.4 三道题的共性复盘

把三道题并排看, 套路完全一致:

	R1 异或	R2 乘加	589 RC4
变换	$x \oplus 0x23$	$(3b + 7) \bmod 256$	KSA+PRGA 密钥流
目标数据在哪	.rodata 的数组	脚本里的 TARGET	.rodata 的 GTK1 标记后
逆运算	再异或一次	乘模逆元 171	同密钥重跑 RC4
独立验证	程序打印 correct	程序 + 正向复算	往返校验 + GATE OPEN

一句话记忆

无论变换多花哨, 解题骨架都是: 找变换 → 写等式 → 求逆 → 喂回原程序。

第五章 复盘：知识地图与易错清单

本章目标：把三道题的经验拧成一张地图，明确“下次拿到逆向题，第一步做什么”。

5.1 知识地图

图 2.4 的五步路线是主干道，本章把它展开成一张知识地图。逆向题的全部知识点挂在五个树杈上：

1. **认文件：**file 看格式与架构（ELF/PE、32/64 位、是否去符号）；readelf -h 看入口与类型（2.x、3.x 节）；
2. **找数据：**strings 找提示语；xxd / objdump -s 看.rodata 常量；“乱码”字符串常是加密的目标数组（图 3.2 与 图 4.4 的呼应就是活例子）；
3. **读逻辑：**objdump -d 反汇编，从成功/失败输出反向追踪比较点；把汇编提炼成“读入、长度、变换、比较”四句话（2.2 节的寄存器、指令与栈帧是读懂它的前提）；
4. **写等式：**把比较写成 $f(input) = target$ ，判断 f 是否可逆；XOR 再异或一次、乘加求模逆元、RC4 重跑一遍（2.5–2.8 节）；
5. **做验证：**结果喂回原程序（correct / GATE OPEN）；正向复算与脚本内置校验交叉印证（图 4.10）。

编译链（图 2.1）解释了“为什么能反汇编”，源码/汇编/机器码对照（图 2.2）解释了“反汇编在读什么”，AT&T 与 Intel 语法的真实对照（图 2.3）是读反汇编的入门钥匙，XOR 流程图（图 2.5）和 RC4 流程图（图 2.6）是两类最常见变换的解剖图。把这六张图钉在墙上，做题时对号入座即可。

5.2 易错清单

下表汇总本册带做中真实出现过（或初学者高频出现）的坑：

坑	症状	解法
把“没搜到 flag”当成无解	strings 输出里没有 flag	目标值常以变换后的字节数组存在，去看 .rodata
凭 Intel 习惯读 AT&T 汇编	参数/搬运方向读反，栈方向想错	先判断语法流派；AT&T 是“源，目的”（第 2.2 节）
复制输入带了空格或换行	字节明明对却 nope	先数长度；用 echo -n 或在脚本里 strip
长度关卡没过就猜算法	怎么算都 nope	先确认长度检查（R1 的 17 字节）
忘记异或是按位运算	手算和脚本对不上	用 python3 算，别心算十六进制
模运算直接做除法	反推结果全错	乘法要乘模逆元（171），不是除以 3
乘数与 256 不互素还套公式	反推出多个候选	先查互素，再决定策略
凭变量名判断密钥/密文	解出乱码	以调用顺序和实际运算为准（557 qgd 的教训）
只跑破解脚本就提交	平台判错或无法复核	必须让原程序/平台确认
照抄讲义地址	自己机器上找不到	地址随编译变化，只抄“结构”不抄数字
在 Windows 里运行 ELF	报“不是可执行文件”	到 WSL 里运行

5.3 下次遇到这类题怎么办

按“有没有源码”“程序能不能跑”分四种情形，各给一条起步路线：

- **有源码的练习题**（如 R1、R2）：读源码写四句话摘要 → 手算首字节 → 写脚本 → 喂回程序。重点练“写等式”，别急着抄答案；
- **只有二进制、能运行**（如 589）：file/strings 建基线 → 喂输入看拒绝信息 → objdump -s 找常量路标（GTK1 这类标记）→ objdump -d 追数据引用 → 脚本求解 → 票根类的中间产物也要喂回程序验证；
- **只有二进制、不能/不敢运行**：全程静态。objdump -s 直接读数据段、objdump -d 读逻辑，用 python 离线解密；陌生 Windows EXE 尤其如此；
- **题面给机器码或字节码**（如 557）：把十六进制文本还原成字节，用反汇编器/字节码工具读指令，翻译成分支与运算，再按索引反推列表。

最后是通用的报告模板（写题解、交接队友都用得上）：材料是什么（文件名 + 哈希）→ 基线是什么（错误输入的输出）→ 证据在哪（命令 + 偏移/地址）→ 变换是什么（公式）→ 答案怎么算（脚本）→ 怎么验证（原程序/平台反馈）。按事实汇报，没证明的事项明确写“尚未证明”。

小技巧

卡住超过 20 分钟就换角度：没看过数据段就去看数据段，没跑过黑盒就去跑黑盒。逆向题的线索几乎总是藏在你还没做的那一步里。

第六章 自测与参考答案

本章目标：先独立作答再对照推演；推演刻意写全中间判断，照抄最后一行命令就错过了训练目标。

6.1 自测题

1. 为什么 strings 没找到 flag 不能说明题目没有 flag?
2. R1 的首个目标字节是 0x45，为什么正确输入的首字符是 f?
3. R1 的长度检查和内容检查各起什么作用？只过其中一关会怎样？
4. R2 的变换 $t = (3b + 7) \bmod 256$ 中，为什么反推要乘 171 而不是除以 3？
5. 若把 R2 的乘数改成 2，为什么无法对所有目标字节唯一还原？
6. $\& 255$ 和 $\% 256$ 有什么关系？为什么说它们对字节运算等价？
7. RC4 的 KSA 和 PRGA 各做什么？解密时要重新执行哪一部分？
8. 589 为什么要先解票根再解 flag？直接解 flag 密文会得到什么？
9. “脚本打印出 flag{...}” 和 “解题完成” 之间还差什么？
10. file 输出里的 stripped 说明什么？对逆向有什么影响？

6.2 参考要点

1. strings 只发现**原样存放**的连续可打印文本；目标值可能经过异或、拆分或编码，藏在.rodata 里。R1 的目标数组在 strings 里只是一串“乱码”（图 3.2）。
2. $0x45 \oplus 0x23 = 0x66$ ，ASCII 0x66 是 f。先手工算首字节可以快速判断方向对不对。
3. 长度检查保证输入是 17 字节（防止长度不同引起的错位比较），内容检查保证每个字节都对。长度不对会直接 nope（图 4.1 第一条），长度对但内容错也会 nope（第二条），两个 nope 的含义不同。

4. 因为计算含模 256，除法不封闭；3 在模 256 下的逆元是 171 ($3 \times 171 \equiv 1$)，乘 171 与“除以 3”等价且唯一。
5. 2 与 256 不互素，不存在模逆元：偶数目标值对应两个可能的输入字节，奇数目标值不可能由该变换产生。
6. 对字节（低 8 位）而言， $x \& 255$ 与 $x \% 256$ 结果相同；前者是位运算写法，后者是算术写法。
7. KSA 用密钥打乱 256 项状态表；PRGA 逐字节生成密钥流。解密时两者都要重新执行（用同一个密钥），再与密文异或一次。
8. 第二次 RC4 的密钥是票根本身；不知道票根就没有正确的初始状态表，直接解 flag 密文只会得到乱码（不会报错）。
9. 还差原程序或平台的确认。破解脚本的输出只是候选答案，必须让原程序打印 correct / GATE OPEN，或平台判定正确。
10. 符号表被剥离，反汇编里函数名消失、只剩地址，定位逻辑更费力；但数据和算法本身不变，字符串路标与数据段分析照常有效。

6.3 完整推演一：R1 从目标数组还原输入

先建立基线：`echo hello | labs/reverse/check1` 输出 `nope`（图 4.1）。读源码（图 4.2）得到判定为 `input[i] ^ 0x23 == expected[i]`，共 17 项。写成等式： $input[i] = expected[i] \oplus 0x23$ 。

手算首项： $0x45 \oplus 0x23 = 0x66 = f$ ，像 flag 开头，方向正确。对 17 项批量计算（图 4.5）得到 `flag{reverse_xor}`。喂回原程序输出 `correct`（图 4.6）。反向验证：把结果逐字节再异或 0x23，还原出原目标数组的 17 个字节。两条独立证据链闭合。

6.4 完整推演二：R2 从目标列表还原输入

黑盒输入只得到 `nope`（图 4.7）。读脚本（图 4.8）抄出变换 $t = (3b + 7) \bmod 256$ 与 20 项目标。先验逆元： $\text{pow}(3, -1, 256) = 171$ （图 4.9）。对首项 57： $(57 - 7) \times 171 = 50 \times 171 = 8550 = 33 \times 256 + 102$ ，得 102 即 `f`。批量反推得 `flag{trace_the_loop}`。

验证分三步：喂回验证器得 `correct`；把答案的 ASCII 码正向做一遍变换，20 个结果逐项等于 `TARGET`（图 4.10）；首字节手工核对通过。若只在脚本里打印字符串而不让验证器接受，就还缺最后一步。

6.5 完整推演三：589 从.rodata 到两行输出

file 确认去符号 ELF（图 4.11）。strings 尾部的 GTK1 是数据路标。objdump -s 读.rodata（图 4.12）：标记 +5 起 16 字节为密钥，+21 起 11 字节为票根密文，402080 处 38 字节为 flag 密文。脚本用标准 RC4 两步（图 4.13）：先解出 11 字节可打印票根 GT-A7K9M2QX 并通过往返自检，再以票根为密钥解出 flag{0a109ca9d704833bc2924f889d1b5576}。票根回喂程序得到 GATE OPEN，是第四重印证。

6.6 保留记录：玄机 557 qgd 的完整推演（原讲义结论原样保留）

本节内容来自原讲义的已验证结论，原样保留，不作改动。题目为玄机 第三届黄河流域公安院校网络安全技能挑战赛 qgd (<https://xj.edisec.net/challenges/557>)，官方 ZIP 含 lookatme.txt、part1flag.txt、part2flag.exe，最终格式为 flag{part1flag/part2flag}。

第一部分：part1flag.txt 前半段是 x86-64 机器码的十六进制文本（全文去除首尾空白后共 695 字符，含空格分隔，对应 232 字节机器码，节选如下），后半段给出加密列表：

```
48 89 4C 24 08 55 57 48 81 EC 28 01 00 00 48 8D 6C 24 20 48 8D 0D 34 08 01 00 E8 96
FB FF FF 48 83 BD 20 01 00 00 00 75 05 E9 BF
```

```
encrypted = [8, 107, 2, 60, 84, 96, 84, 111, 84, 104]
part1 = bytes(x ^ (0x31 if i % 2 == 0 else 0x58)
               for i, x in enumerate(encrypted))
print(part1) # b'933de8e7e0'
```

反汇编可见程序用位置加一后的奇偶性分支：原位置为偶数时异或 0x31，奇数时异或 0x58。例如第零项 $8 \oplus 49 = 57$ 即 9，第一项 $107 \oplus 88 = 51$ 即 3。完整第一段为 933de8e7e0。知识点：读机器码时要把分支条件翻成“原列表索引的奇偶”，避免把两把异或密钥用反。

第二部分：part2flag.exe 是 PyInstaller 打包的 Python 程序。静态提取 Python 3.9 字节码后，入口调用 decrypt(ciphertext, key)；长十六进制常量经 bytes.fromhex 后实际作为密钥，短字节常量虽变量名叫 ciphertext，实际作为密文——**变量名不是算法依据，调用顺序才是**。其算法与标准 RC4 相似却不相同：状态表只有 128 项；初始化每轮对索引异或 55；输出字节由状态值的高低四位组合 $((16t)|(t \gg 4)) \& 255$ 。因此直接调用现成 RC4 库会得到错误文本。按实际代码还原后得到第二段：

```
3b162f64bcfec94803d
```

两段按题面格式组合：

flag: flag{933de8e7e0/3b162f64bcfec94803d}

此答案已在平台提交，页面显示“FLAG 正确”和步骤 1/1。讲义附带脚本 `python3 labs/platform/solve_qgd.py path/to/qgd.zip` 保存了静态提取的常量，并核对官方 ZIP 中两份材料的 SHA-256，防止误用不同版本附件；脚本不运行 EXE。

6.7 自测评分参考

10 题中答对 8 题以上，说明本册知识地图已建立，可以去做平台题；答对 5–7 题，建议重读第 2 章对应小节并重做该题的变式练习；少于 5 题，先按第 1 章把环境与命令行练熟，再从 R1 的黑盒步骤重新走一遍。

附录 A 命令速查表

本章目标：忘记命令时来这里查：左列是想做什么，右列是敲什么。所有命令默认在 WSL 的资料根目录执行。

A.1 观察与识别

想做什么	敲什么命令
确认当前目录	<code>pwd; ls</code>
识别文件类型/架构	<code>LC_ALL=C file 目标文件</code>
找可打印字符串	<code>strings -a 目标文件</code>
看文件头字节	<code>xxd -l 48 目标文件</code>
看指定偏移的字节	<code>xxd -s 0x2010 -l 32 目标文件</code>
看 ELF 头（架构/入口）	<code>LC_ALL=C readelf -h 目标文件</code>
看 ELF 分段表	<code>LC_ALL=C readelf -S 目标文件</code>
看符号表	<code>nm 目标文件</code>

A.2 反汇编与数据段

想做什么	敲什么命令
反汇编全部代码	<code>LC_ALL=C objdump -d 目标文件</code>
反汇编（Intel 语法）	<code>LC_ALL=C objdump -d -M intel 目标文件</code>
只看 main 开头	<code>LC_ALL=C objdump -d 目标文件 sed -n '/<main>:/,+20p'</code>
按地址切片	<code>LC_ALL=C objdump -d 目标文件 sed -n '/126a:/,/12b1:/p'</code>
读数据段字节	<code>LC_ALL=C objdump -s -j .rodata 目标文件</code>
在反汇编里找常量	<code>objdump -d 目标文件 grep 0x23</code>
记录文件哈希	<code>sha256sum 目标文件</code>

A.3 运行与破解

想做什么	敲什么命令
喂输入给程序（失败演示）	<code>echo hello labs/reverse/check1</code>
喂正确输入验证	<code>echo flag{reverse_xor} labs/reverse/check1</code>
运行 Python 验证器	<code>echo flag{trace_the_loop} python3 labs/reverse/verify2.py</code>
单行异或反推	<code>python3 -c 'print(bytes(x ^ 0x23 for x in E).decode())'</code>
求模逆元	<code>python3 -c 'print(pow(3, -1, 256))'</code>
乘加变换反推	<code>python3 -c 'print(bytes(((t - 7) * 171) & 255 for t in T).decode())'</code>
跑 589 解题脚本	<code>python3 labs/platform/solve_589_gate_ ticket.py</code>
编译练习程序	<code>gcc -O0 -o labs/reverse/check1 labs/ reverse/check1.c</code>
看上条命令退出码	<code>echo \$?</code>

附录 B 术语表

本章目标：按字母顺序速查；每个术语一句话解释，正文首现处有展开。

- **ASCII：**数字与字符的对应表，如 102 对应 f；`chr()`/`ord()` 互转。
- **AT&T / Intel 语法：**汇编的两种书写流派；objdump 默认 AT&T（操作数“源, 目的”），加 `-M intel` 切换为 Intel（“目的, 源”）。
- **base64：**用 64 个可打印字符编码任意字节的记法，常见于嵌入文本的数据（本册题目未用到，Misc 方向常见）。
- **字节 (byte)：**8 位二进制，取值 0–255，计算机存储的最小单位。
- **反汇编 (disassembly)：**把机器码翻译回汇编指令，工具如 `objdump -d`。
- **ELF：**Linux 可执行文件格式 (Executable and Linkable Format)，与 Windows 的 PE 对应。
- **entry point (入口点)：**程序开始执行的第一条指令地址，`readelf -h` 可见。
- **fgets：**C 标准库函数，从标准输入读一行文本。
- **Ghidra / IDA：**图形化反编译工具，把汇编还原成伪代码；伪代码的变量名是推测，细节以汇编为准。
- **十六进制 (hex)：**两位数字表示一个字节的记法，0x 前缀标记。
- **黑盒 / 白盒：**只看输入输出的观察方式 / 读内部实现的分析方式。
- **KSA：**RC4 的密钥调度阶段，用密钥打乱 256 项状态表。
- **机器码 (machine code)：**CPU 直接执行的字节序列。
- **基线 (baseline)：**正常/失败状态下的输出参照，用于判断变化。
- **汇编 (assembly)：**机器码的助记文本，如 `xor $0x23,%eax`。
- **寄存器 (register)：**CPU 内部直接参与运算的小块存储；整型参数按 rdi、rsi、rdx、rcx 传，返回值在 rax。

- **立即数** (immediate): 直接写在指令里的常数, AT&T 里带 \$ 前缀, 如 \$0x23。
- **链接** (linking): 把目标文件与库函数拼成可执行文件的步骤。
- **流密码** (stream cipher): 逐字节用密钥流加密的密码体制, RC4 是代表。
- **魔数** (magic number): 标识文件格式的固定字节序列, 如 ELF 的 7f 45 4c 46。
- **模逆元** (modular inverse): 满足 $a \cdot a^{-1} \equiv 1 \pmod{m}$ 的数; 3 对 256 的模逆元是 171。
- **PIE**: 位置无关可执行文件, 加载地址随机; readelf -h 显示类型 DYN。
- **管道** (pipe): |, 把左边命令的输出接给右边命令作输入。
- **PRGA**: RC4 的密钥流生成阶段, 每输出一字节搅动一次状态表。
- **payload**: 为触发目标行为而构造的输入数据 (本册的“正确输入”即一种 payload)。
- **伪代码** (pseudo-code): 反编译工具生成的类 C 代码, 便于阅读但非原始源码。
- **RC4**: 经典流密码, KSA 打乱状态表、PRGA 生成密钥流, 逐字节异或加解密。
- **.rodata**: ELF 的只读数据段, 常量数组与提示字符串常在此。
- **符号表** (symbol table): 函数名与地址的对应表; 被剥离 (stripped) 后只剩地址。
- **标准输入/输出** (stdin/stdout): 程序默认的读入/打印通道, 可被管道替换。
- **十六进制文本**: 把字节写成十六进制字符串的形式 (如 48 89 4c), 机器码的常见出题形式。
- **退出码** (exit code): 命令结束时返回的整数, 0 表示成功。
- **.text**: ELF 的代码段, 存放机器码。
- **票根** (ticket): 589 题中 11 字节的通关凭证, 兼作第二步 RC4 的密钥。
- **异或** (XOR): 按位“相异为 1”的运算, 自己是自己的逆。
- **掩码** (mask): 用按位与只保留想要的位, 如 $x \& 0xFF$ 只留最低 8 位。
- **移位** (shift): << / >> 把二进制整体左移 / 右移, 相当于乘 / 除 2^n 。
- **栈** (stack): 函数调用的“后进先出”内存区, 从高地址向低地址生长, rsp 指向栈顶。
- **栈帧** (stack frame): 一个函数在栈上占用的那一段, rbp 指向其起点; 局部变量按 rbp-偏移访问。
- **调用约定** (calling convention): 函数传参与返回值的规则; x86-64 前 6 个整型参数走 rdi/rsi/rdx/rcx/r8/r9, 返回值在 rax。
- **正向复算**: 用还原结果重跑一遍原变换、比对目标值的验证手法。

- **字节流** (byte stream): 连续的字节序列; RC4 的密钥流即一串伪随机字节。